

Projet : Analyse numérique avec C++

Duc Hoang Viet NGUYEN

duc-hoang-viet.nguyen.etu@univ-lille.fr

Parcours ISN

18 octobre 2025

Table des matières

1	Introduction	2
2	Résolution du problème de Poisson 1D	2
2.1	Rangement profil	2
2.2	Factorisation de $\text{LDLt}(A)$ et Résolution de $\text{LDLt}(A,b)$	4
2.3	Vérification de Poisson	6
2.4	Tests	7
2.5	Graphe	9
3	Schémas numériques pour l'équation de la chaleur	12
3.1	Condition initiale, Condition au bord	12
3.2	Chaleur explicite, Chaleur implicite	14
3.3	Vérification de chaleur	21
3.4	Tests	23
3.5	Graphe	25
4	Résolution de l'équation de Black-Scholes	26
4.1	Initialisation	27
4.2	Matrice bande pour Black-Scholes	27
4.3	Méthode Simpl pour Black-Sholes	28
4.4	Tests	30
4.5	Graphe	31
4.5.1	Graphe en 3D	31
4.5.2	Graphe en Heat Map	34
5	Méthodes itératives pour les matrices creuses : application à la température optimale d'un four pour la cuisson d'un gâteau	35
5.1	Première partie :La discrétisation par différences finies	36
5.2	Deuxième parite : La méthode de gradient à pas optimal et à pas fixe. . . .	46
5.3	Troisième partie : la méthode PCG de gradient conjugué avec préconditionnement	48
5.4	Quatrième partie(Bonus) : optimiser la cuisson du gâteau	53
6	Code	56

1 Introduction

Ce projet a pour objectif d'appliquer les méthodes d'analyse numérique en C++ à la résolution de problèmes issus des mathématiques appliquées, de la physique et de la finance. Il commence par la résolution de l'équation de Poisson 1D à l'aide de matrices profil et de la factorisation LDL, avec validation numérique des résultats. Les équations de la chaleur sont ensuite étudiées via des schémas explicites et implicites, comparés à l'aide de simulations et de représentations graphiques. Le modèle de Black-Scholes est traité numériquement par la construction de matrices bandes et la visualisation des solutions en 2D, 3D et cartes de chaleur. Enfin, des méthodes itératives pour matrices creuses sont appliquées à l'étude stationnaire du champ de température d'un gâteau dans un four.

2 Résolution du problème de Poisson 1D

Soit $\Omega =]0, 1[$ et on cherche une fonction $u : \Omega \rightarrow \mathbb{R}$ telle que

$$\begin{cases} -u''(x) + \alpha u(x) = f(x), & \forall x \in \Omega, \\ u(0) = u(1), \end{cases} \quad (1)$$

avec $\alpha \geq 0$ et f donnée.

Pour discrétiser le problème par la méthode des différences finies, on introduit un maillage de $N + 1$ points équidistants, dont $N - 1$ sont à l'intérieur de l'intervalle $]0, 1[$:

$$0 = x_0 < x_1 < \dots < x_{N-1} < x_N = 1, \quad h = \frac{1}{N}, \quad x_i = ih, \quad i = 0, \dots, N.$$

On note u_i une approximation de $u(x_i)$. Le système linéaire obtenu est :

$$\begin{cases} -\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} + \alpha u_i = f_i, & i = 0, \dots, N-1, \\ u_0 = u_N. \end{cases} \quad (2)$$

2.1 Rangement profil

On écrit la fonction `matrice_1D_profil(N, c0, c1)` qui donne en sortie la matrice A du problème de Poisson en stockage profil, avec N sa dimension, $c1$ le coefficient sur la sous-diagonale et $c0$ celui sur la diagonale. Faire attention à la forme de la matrice liée aux conditions aux limites périodiques.

On considère la fonction exacte $u_{\text{ex}}(x) = \sin(2\pi x)$, qui est périodique, donc $u_0 = u_N$. On pose alors :

$$c_0 = \frac{2}{h^2} + \alpha, \quad c_1 = -\frac{1}{h^2},$$

et la résolution du problème (1) conduit à la résolution du système linéaire

$$AU = F, \quad \text{avec } A \in \mathcal{M}_N(\mathbb{R}), \quad U = \begin{pmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{pmatrix}.$$

La matrice A est donnée par :

$$A = \begin{pmatrix} \frac{2}{h^2} + \alpha & -\frac{1}{h^2} & 0 & \cdots & 0 & -\frac{1}{h^2} \\ -\frac{1}{h^2} & \frac{2}{h^2} + \alpha & -\frac{1}{h^2} & \cdots & 0 & 0 \\ 0 & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & -\frac{1}{h^2} & \frac{2}{h^2} + \alpha & -\frac{1}{h^2} \\ -\frac{1}{h^2} & 0 & \cdots & 0 & -\frac{1}{h^2} & \frac{2}{h^2} + \alpha \end{pmatrix}$$

et le second membre :

$$F = \begin{pmatrix} f_0 \\ f_1 \\ \vdots \\ f_{N-1} \end{pmatrix}.$$

On peut considérer le stockage profil de A :

$$\text{ATAB} = \left(\frac{2}{h^2} + \alpha, -\frac{1}{h^2}, \frac{2}{h^2} + \alpha, \dots, -\frac{1}{h^2}, \frac{2}{h^2} + \alpha \right), \quad \text{INDIAG} = (0, 2, 4, \dots, 2k+N).$$

Le vecteur **ATAB** contient les coefficients du profil de la matrice (les valeurs diagonales et sous-diagonales successives), tandis que **INDIAG** stocke les positions des éléments diagonaux principaux. Ainsi, chaque ligne i du profil commence à l'indice **INDIAG**[i] dans le tableau **ATAB**. On souhaite construire la matrice tridiagonale associée au problème de

Poisson en utilisant un **stockage en profil** afin d'économiser la mémoire. Le principe consiste à stocker uniquement les éléments utiles (la diagonale principale et les sous/sur-diagonales).

```

1 struct mat_profil {
2     vector<int> INDIAG;    // indices de la diagonale principale
3     vector<double> ATAB;  // tableau contenant les coefficients utiles
4 };
5
6 mat_profil matrice_1D_profil(int N, double c0, double c1) {
7     mat_profil A;
8     A.INDIAG.resize(N);
9     A.ATAB.resize(3 * N); // taille provisoire
10
11     int k = 0;
12
13     // Premi re ligne
14     A.ATAB[k++] = c0;          // coefficient diagonal
15     A.INDIAG[0] = 0;
16
17     // Lignes suivantes
18     for (int i = 1; i < N - 1; i++) {
19         A.ATAB[k++] = c1;      // sous-diagonale
20         A.ATAB[k++] = c0;      // diagonale principale
21         A.INDIAG[i] = k - 1;   // position de la diagonale principale
22     }
23
24     // Derni re ligne
25     A.ATAB[k++] = c1;

```

```

26   for (int i = 1; i < N - 2; i++) {
27       A.ATAB[k++] = 0;
28   }
29   A.ATAB[k++] = c1;
30   A.ATAB[k++] = c0;
31   A.INDIAG[N - 1] = A.INDIAG[N - 2] + N;
32   A.ATAB.resize(k);
33   return A;
34 }

```

Listing 1 – Construction de la matrice 1D en stockage profil

Dans ce code, on construit pas à pas le **profil** de la matrice A :

- La structure `mat_profil` contient deux vecteurs :
 - `INDIAG` stocke la position de chaque élément diagonal dans le tableau compact `ATAB`.
 - `ATAB` contient tous les coefficients utiles (diagonale principale et sous-diagonale).
- On commence par remplir la première ligne : la diagonale principale vaut c_0 .
- Pour chaque ligne suivante, on ajoute d'abord le coefficient c_1 (sous-diagonale), puis c_0 (diagonale principale).
- Le tableau `INDIAG[i]` garde la trace de la position du coefficient diagonal correspondant à la ligne i .
- Enfin, la dernière ligne est traitée séparément pour respecter les conditions aux limites périodiques ($u_0 = u_N$).
- Pour cette dernière ligne :
 - On commence par placer le premier coefficient c_1 .
 - Ensuite, de l'indice 1 jusqu'à $N - 2$, on parcourt une boucle où chaque élément est initialisé à 0.
 - Puis, on ajoute de nouveau le coefficient c_1 à la position $N - 2$ et enfin c_0 sur la diagonale principale (position $N - 1$).
 - On met ensuite à jour l'indice de la diagonale principale correspondante :

$$\text{INDIAG}[N-1] = \text{INDIAG}[N-2] + N.$$

Cela permet de repérer correctement la position du dernier élément diagonal dans le tableau `ATAB`.

Ainsi, le tableau `ATAB` représente de façon compacte la matrice tridiagonale :

$$A = \begin{pmatrix} c_0 & c_1 & 0 & \cdots & 0 & c_1 \\ c_1 & c_0 & c_1 & \cdots & 0 & 0 \\ 0 & \ddots & \ddots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & c_1 & c_0 & c_1 \\ c_1 & 0 & \cdots & 0 & c_1 & c_0 \end{pmatrix}.$$

2.2 Factorisation de $\text{LDLt}(A)$ et Résolution de $\text{LDLt}(A,b)$

Les fonctions `factorisation_LDLt(A)` et `resolution_LDLt(A,b)` développées au TP3 permettent de résoudre le système d'équations $Ax = b$. Pour valider toutes ces fonctions, on introduit d'abord la fonction exacte $u_{\text{ex}}(x) = \sin(2\pi x)$ et on calcule (à la main) le second membre de l'équation $f(x) = (-u_{\text{ex}}'' + \alpha u_{\text{ex}})(x)$.

Considère $u_{\text{ex}}(x) = \sin(2\pi x)$. Calculons ses dérivées :

$$u'_{\text{ex}}(x) = 2\pi \cos(2\pi x), \quad u''_{\text{ex}}(x) = -(2\pi)^2 \sin(2\pi x) = -4\pi^2 \sin(2\pi x).$$

Le second membre de l'équation est donné par

$$f(x) = (-u''_{\text{ex}} + \alpha u_{\text{ex}})(x).$$

En remplaçant u''_{ex} et u_{ex} , on obtient

$$f(x) = -(-4\pi^2 \sin(2\pi x)) + \alpha \sin(2\pi x) = (4\pi^2 + \alpha) \sin(2\pi x).$$

Ainsi,

$$f(x) = (4\pi^2 + \alpha) \sin(2\pi x). \quad (3)$$

Remarque : u_{ex} est 1-périodique, donc satisfait bien la condition périodique $u_0 = u_N$.
Ce sont les fonctions `factorisation_LDLt(A)` et `resolution_LDLt(A,b)` développées au TP3, qui permettent de résoudre le système d'équations $Ax = b$.

```

1 void factorisation_LDLt(mat_profil& A){
2
3     int n = A.INDIAG.size();
4     for (int i = 0; i < n; i++) {
5         int ji;
6         if (i == 0) ji = 0;
7         else ji = i - A.INDIAG[i] + A.INDIAG[i-1] + 1;
8
9         for (int j = ji; j <= i - 1; j++) {
10            int jj;
11            if (j == 0) jj = 0;
12            else jj = j - A.INDIAG[j] + A.INDIAG[j-1] + 1;
13
14            // etape 5 7
15            for (int k = max(ji, jj); k <= j - 1; k++) {
16                A.ATAB[A.INDIAG[i] - i + j] -= A.ATAB[A.INDIAG[i] - i +
17                k] * A.ATAB[A.INDIAG[j] - j + k];
18            }
19            // etape 9 13
20            double S = A.ATAB[A.INDIAG[i] - i + j] / A.ATAB[A.INDIAG[j
21            ]];
22            A.ATAB[A.INDIAG[i]] -= S * A.ATAB[A.INDIAG[i] - i + j];
23            A.ATAB[A.INDIAG[i] - i + j] = S;
24        }
25    } // A maintenant contient L et D (LDLt)
26 }
27
28 template <typename T>
29 vector<T> resoudre_LDLt(const mat_profil& A, const vector<T>& b) {
30     int n = A.INDIAG.size();
31     assert((int)b.size() == n);
32
33     vector<T> x = b;
34
35     for (int i = 1; i < n; i++) {
36         int ji = i - A.INDIAG[i] + A.INDIAG[i-1] + 1;
37         for (int j = ji; j <= i-1; j++) {
38             x[i] -= A.ATAB[A.INDIAG[i] - i + j] * x[j];
39         }
40     }
41     return x;
42 }
```

```

37     }
38 }
39 for (int i = 0; i < n; i++) {
40     x[i] /= A.ATAB[A.INDIAG[i]];
41 }
42 for (int i = n-1; i >= 1; i--) {
43     int ji = i - A.INDIAG[i] + A.INDIAG[i-1] + 1;
44     for (int j = ji; j <= i-1; j++) {
45         x[j] -= A.ATAB[A.INDIAG[i] - i + j] * x[i];
46     }
47 }
48 return x;
49 }

```

2.3 Vérification de Poisson

On écrit la fonction `verif_poisson(N, file)` qui résout le problème de Poisson avec cette fonction f . On affiche N et l'erreur $\varepsilon_N = \max_j |u_j - u_{\text{ex}}(x_j)|$ à l'écran et dans un fichier "file".

```

1 template <typename T>
2 void verif_poisson(const mat_profil& A, const vector<T>& b, int N, double h
3     , double alpha, const string& file) {
4
5     mat_profil Acopy = A;
6     factorisation_LDLt(Acopy);
7     vector<double> x = resoudre_LDLt(Acopy, b);
8
9     // cout << "i \t x_i \t Approx \t exact \t erreur" << endl;
10    double erreur_max = 0.0;
11    for (int i = 0; i < N; i++) {
12        double x_exact = sin(2*M_PI*i*h);
13        double err = fabs(x[i] - x_exact);
14        erreur_max = max(erreur_max, err);
15        // cout << i << "\t" << i*h << " \t " << x[i] << " \t " <<
16        x_exact << " \t " << err << endl;
17    }
18    cout << "Erreur maximale = " << erreur_max << endl;
19
20    // criture dans un fichier
21    ofstream fout(file, ios::app); // append
22    if (fout.is_open()) {
23        fout << N << "\t" << erreur_max << endl;
24        fout.close();
25    } else {
26        cerr << "Erreur: impossible d'ouvrir le fichier " << file <<
27        endl;
28    }
29 }

```

La fonction `verif_poisson` permet de vérifier la précision de la solution numérique pour le problème de Poisson en 1D. Voici le fonctionnement détaillé :

— **Entrées :**

- **A** : matrice du problème en stockage profil (`mat_profil`).
- **b** : vecteur second membre du système $AU = F$.
- **N** : nombre de points de la grille.

- `h` : pas de discrétisation.
- `alpha` : coefficient du problème de Poisson.
- `file` : nom du fichier dans lequel enregistrer les erreurs.
- **Traitement :**
 1. On copie la matrice `A` dans `Acopy` pour ne pas modifier l'originale.
 2. On effectue la factorisation LDL^T de `Acopy` grâce à `factorisation_LDLt`.
 3. On résout le système linéaire $AU = F$ avec `resoudre_LDLt` et on stocke la solution dans le vecteur `x`.
 4. On calcule l'erreur maximale $\varepsilon_{\max}$ entre la solution numérique x_i et la solution exacte $u_{\text{ex}}(x_i) = \sin(2\pi x_i)$:
$$\varepsilon_{\max} = \max_{0 \leq i < N} |x_i - u_{\text{ex}}(x_i)|.$$
 5. On affiche $\varepsilon_{\max}$ à l'écran.
 6. On écrit `N` et $\varepsilon_{\max}$ dans le fichier `file` en mode ajout (`append`).
- **Remarques :**
 - La solution exacte $u_{\text{ex}}(x) = \sin(2\pi x)$ est périodique, donc $u_0 = u_N$.
 - Si l'ouverture du fichier échoue, un message d'erreur est affiché sur la sortie d'erreur standard.

2.4 Tests

Le programme principal exécutera `verif_poisson` pour un α de votre choix, en prenant $N = \{4, 8, 16, 32, 64, 128, 256, \dots\}$. On sauvera les résultats dans un fichier, où chaque ligne contiendra N et ε_N . Que se passe-t-il si on prend $\alpha = 0$?

```

1 int main() {
2     double alpha = 0.0;
3     string fichier = "erreur.txt";
4     ofstream fout(fichier);
5     fout << "N \tErreur_max s\n";
6     fout.close();
7     // Tests pour diff rentes tailles de matrice
8     vector<int> Ns = {4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096,
9     8192, 16384, 32768, 65536};
10    for (int N : Ns) {
11        double h = 1.0 / N;
12        double c0 = 2.0/(h*h) + alpha;
13        double c1 = -1.0/(h*h);
14
15        mat_profil A = matrice_1D_profil(N, c0, c1);
16        vector<double> b(N);
17        for (int i = 0; i < N; i++) {
18            double x_i = i*h;
19            b[i] = (4*M_PI*M_PI + alpha) * sin(2*M_PI*x_i);
20        }
21        cout << "R solution pour N = " << N << endl;
22        verif_poisson(A,b, N,h, alpha, fichier);
23    }
24    cout << "\nR sultats sauvegard s dans : " << fichier << endl;
25    return 0;
26 }
```

Le programme principal exécute les tests pour différentes tailles de matrice et sauvegarde les résultats dans un fichier. Voici le fonctionnement détaillé ligne par ligne :

- **Initialisation :**
 - `double alpha = 0.0;` : choix du coefficient α pour le problème de Poisson.
 - `string fichier = "erreur.txt";` : nom du fichier où seront enregistrés les résultats.
 - `ofstream fout(fichier);` et `fout << "N_max"; fout.close();` : création du fichier et écriture de l'en-tête.
- **Définition des tailles de matrice :** `vector<int> Ns = {4, 8, 16, 32, 64, 128, ..., 65536};` Ce vecteur contient toutes les valeurs de N pour lesquelles on souhaite tester la fonction `verif_poisson`.
- **Boucle sur toutes les tailles N :**
 1. Calcul du pas de discrétisation : $h = 1/N$.
 2. Définition des coefficients de la matrice :
$$c_0 = \frac{2}{h^2} + \alpha, \quad c_1 = -\frac{1}{h^2}.$$
 3. Construction de la matrice profilée `A` avec `matrice_1D_profil(N, c0, c1)`.
 4. Construction du second membre `b` : pour chaque $i = 0, \dots, N - 1$,
$$x_i = i \cdot h, \quad b_i = (4\pi^2 + \alpha) \sin(2\pi x_i).$$
 5. Affichage du message "Résolution pour N = ..." pour indiquer la taille actuelle.
 6. Appel de `verif_poisson(A, b, N, h, alpha, fichier)` pour résoudre le système et sauvegarder l'erreur maximale.
- **Fin du programme :** Affichage du message indiquant que tous les résultats ont été sauvegardés dans le fichier `erreur.txt`, puis retour de 0.

Remarque : Ce programme permet de tester l'évolution de l'erreur maximale ε_N lorsque la taille de la grille N augmente, et de sauvegarder ces résultats pour analyse.

Voici les résultats obtenus lors de l'exécution du programme :

Pour $\alpha = 10.0$

```

1 R solution pour N = 4
2 Erreur maximale = 0.37389
3 R solution pour N = 8
4 Erreur maximale = 0.338671
5 R solution pour N = 16
6 Erreur maximale = 0.255195
7 R solution pour N = 32
8 Erreur maximale = 0.158222
9 R solution pour N = 64
10 Erreur maximale = 0.0881403

```

Pour $\alpha = 0.0$

```

1 R solution pour N = 4
2 Erreur maximale = 0.644413
3 R solution pour N = 8
4 Erreur maximale = 0.482432

```

```

5 R solution pour N = 16
6 Erreur maximale = 0.306386
7 R solution pour N = 32
8 Erreur maximale = 0.173318
9 R solution pour N = 64
10 Erreur maximale = 0.0922294

```

Conclusion sur la convergence et l'effet de α

Lorsque le nombre de points de la grille N augmente, le pas de discrétisation $h = 1/N$ diminue, et la solution approchée u_i obtenue par la méthode des différences finies tend vers la solution exacte $u(x)$. Cela se traduit par une diminution de l'erreur maximale ε_N avec N :

$$\varepsilon_N = \max_j |u_j - u_{\text{ex}}(x_j)| \longrightarrow 0 \quad \text{lorsque } N \rightarrow \infty.$$

Conclusion : La méthode des différences finies utilisée pour résoudre le problème de Poisson 1D est convergente.

- Pour $\alpha = 10 > 0$: la matrice du système linéaire est définie positive, donc le système est mieux conditionné, la résolution est stable et l'erreur plus petite.
- Pour $\alpha = 0$: l'opérateur $-u''$ possède un vecteur propre nul, donc la matrice discrète est singulière et le système est mal conditionné. Cependant, si la fonction $f(x)$ vérifie la condition de compatibilité (moyenne nulle), la solution peut quand même être obtenue, mais l'erreur est plus grande et la convergence plus lente.

$\Rightarrow$ Les résultats numériques montrent que lorsque N augmente, l'erreur ε_N diminue, ce qui confirme la convergence de la méthode des différences finies. De plus, les erreurs obtenues avec $\alpha = 10$ sont plus petites que celles avec $\alpha = 0$, car pour $\alpha > 0$, l'opérateur est défini positif et le problème est mieux conditionné.

2.5 Graphe

Avec Python, on peut tracer le nuage de points $(\log(N), \log(\varepsilon_N))$ et calculer la droite de régression afin de déterminer expérimentalement le taux de convergence β dans $\varepsilon_N \approx C h_N^\beta$.

Posons $y = \log(\varepsilon_N)$ et $x = \log(N)$. La relation $\varepsilon_N \approx C h_N^\beta$ est équivalente à

$$\log(\varepsilon_N) = \log(C) + \beta \log(h_N).$$

Or, $h_N = 1/N$ donc $\log(h_N) = -\log(N)$. En substituant, on obtient

$$\log(\varepsilon_N) = -\beta \log(N) + \log(C),$$

c'est-à-dire que l'équation de la droite de régression est

$$y = -\beta x + \log(C).$$

Voici le code en Python pour tracer le graph.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def plot_erreur(file_path="erreur.txt"):
5     data = np.loadtxt(file_path, skiprows=1) # ignorant le titre
6     N = data[:,0]
7     Erreur_max = data[:,1]

```

```

8
9 # Logarithmes des donn es
10 x = np.log(N)
11 y = np.log(Erreur_max)
12
13 # R gression lin aire sur les donn es log-log
14 a, b = np.polyfit(x, y, 1) # y = a*x + b
15 beta = -a # car y = log( ) = - *log(N) + log(C)
16 C = np.exp(b)
17
18 print(f"Taux de convergence exp rimental {beta:.3f}")
19 print(f"Constante C {C:.3e}")
20
21 # Tracer les donn es et la r gression lin aire
22 plt.figure(figsize=(8,5))
23 plt.scatter(x, y, color='blue', label='Donn es (log(N),log( N ))')
24 plt.plot(x, a*x + b, color='red', label=f"R gression lin aire : y
= {a:.3f}x + {b:.3f}")
25 plt.xlabel("log(N)")
26 plt.ylabel("log( N )")
27 plt.title("Convergence exp rimentale du sch ma Poisson 1D")
28 plt.grid(True , which="both", ls="--")
29 plt.legend()
30 plt.show()
31
32 plot_erreur("erreur.txt")

```

Listing 2 – Tracer le nuage de points et calculer la régression linéaire

Explications du code :

- `np.loadtxt(file_path, skiprows=1)` : lit les données numériques depuis le fichier en ignorant la première ligne de titre.
- `N = data[:,0]` et `Erreur_max = data[:,1]` : récupèrent respectivement les tailles de grille et les erreurs maximales.
- Les vecteurs `x` et `y` contiennent les logarithmes des données pour réaliser une régression log-log.
- `np.polyfit(x, y, 1)` : effectue la régression linéaire pour déterminer les coefficients a et b de la droite $y \approx ax + b$.
- β est obtenu comme $\beta = -a$ en raison de la relation $\log(\varepsilon_N) = -\beta \log(N) + \log(C)$, et $C = \exp(b)$.
- Les instructions `plt.scatter` et `plt.plot` tracent respectivement les points expérimentaux et la droite de régression.
- `plt.show()` affiche la figure finale.

Comparaison de la convergence expérimentale pour deux valeurs de α

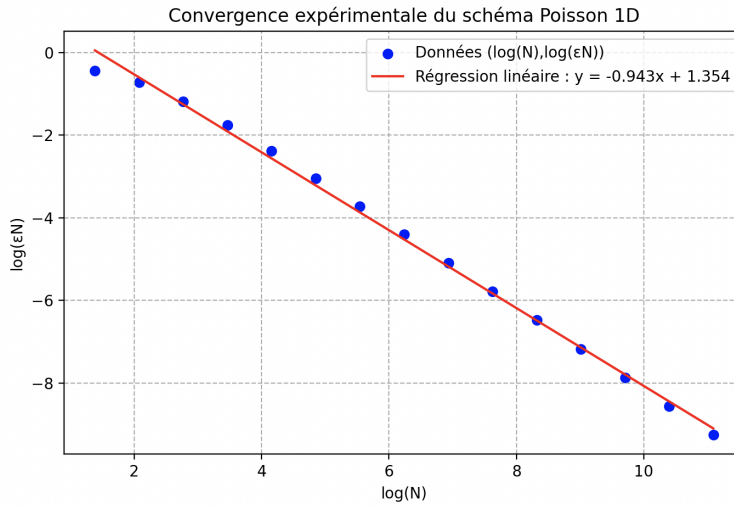


FIGURE 1 – Résultats pour $\alpha = 0$

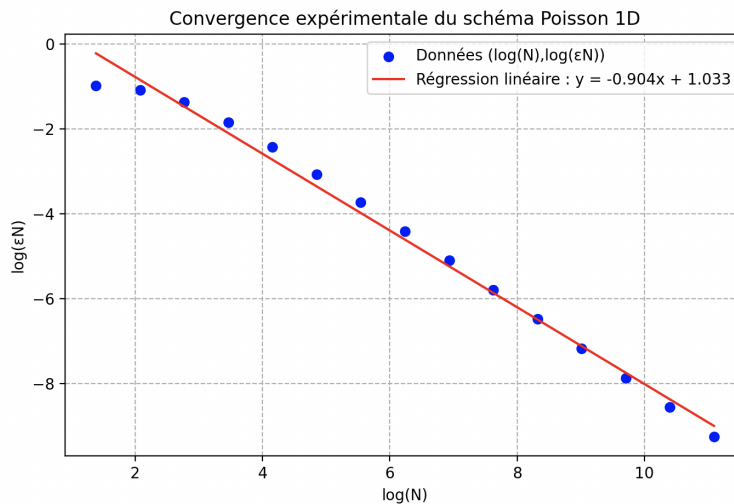


FIGURE 2 – Résultats pour $\alpha = 10$

Les taux de convergence expérimentaux et les constantes obtenus sont :

- Pour $\alpha = 0.0$: $\beta \approx 0.943$, $C \approx 3.874 \times 10^0$
- Pour $\alpha = 10$: $\beta \approx 0.904$, $C \approx 2.808 \times 10^0$

Observations :

- Pour $\alpha = 0$, les points sont **plus alignés** sur la droite de régression, ce qui reflète une évolution plus régulière de l'erreur avec N pour le Laplacien pur.
- Pour $\alpha = 10$, les points sont légèrement moins alignés, car le terme αu influence davantage la solution à petites valeurs de N , perturbant la linéarité apparente dans le graphique log-log.
- La constante C est plus grande pour $\alpha = 0$ (387.4) que pour $\alpha = 10$ (280.8), ce qui correspond à des erreurs initiales plus élevées pour le Laplacien pur.

- L'ordre expérimental β observé est inférieur à l'ordre théorique attendu $\beta = 2$ pour un schéma centré d'ordre 2. Cela s'explique par le fait que l'on considère des valeurs de N relativement petites (non-asymptotiques). Pour des N plus grands, la pente β devrait s'approcher de 2 dans les deux cas.

Conclusion :

- Le schéma reste d'ordre 2 asymptotiquement pour $\alpha = 0$ et $\alpha > 0$.
- La différence d'alignement et de constante C observée expérimentalement est due aux effets de la taille de N et du terme α sur la solution à petites résolutions.
- En résumé, le schéma aux différences finies centrées est d'ordre 2 asymptotiquement pour $\alpha = 0$ et $\alpha > 0$. Les différences observées dans l'alignement des points et la valeur de la constante C sont dues à l'effet de la taille de N et de la présence du terme α sur les solutions pour des résolutions relativement faibles.

3 Schémas numériques pour l'équation de la chaleur

Soit $\Omega =]0, 1[$ et $u_0 \in L^2(\Omega) \cap L^\infty(\Omega)$. On s'intéresse à la résolution numérique par différences finies de l'équation de la chaleur donnée par :

$$\begin{cases} \frac{\partial u(t,x)}{\partial t} - \nu(x) \frac{\partial^2 u(t,x)}{\partial x^2} = f(t,x), & \forall (t,x) \in]0, T] \times]0, 1[, \\ u(0,x) = u_0(x), & \forall x \in]0, 1[, \\ u(t,0) = g(t,0), & \forall t \in [0, T], \\ u(t,1) = g(t,1), & \forall t \in [0, T], \end{cases} \quad (4)$$

avec $\nu(x) > 0$ une fonction qui donne la conductivité thermique de la barre Ω . Ici, on prendra :

$$\nu(x) = \begin{cases} 1, & x \in]0, 0.5[, \\ 10, & x \in [0.5, 1[, \end{cases} \quad (5)$$

qui correspond à une barre formée de deux matériaux de conductivité différente. On va mettre en œuvre les schémas en temps d'Euler explicite (Sexpl) et Euler implicite (Simpl) (donnés en cours) afin de comparer leur précision avec une solution exacte donnée, ainsi que leur rapidité de calcul. Si on note h le pas d'espace et k le pas de temps, on peut montrer que (Sexpl) est stable sous la condition

$$k < \frac{h^2}{2 \|\nu\|_\infty},$$

tandis que (Simpl) est toujours stable (donc on prendra par exemple $k = h$).

3.1 Condition initiale, Condition au bord

On considère la solution exacte $u_{\text{ex}}(t,x) = \cos(t) \cos(\pi x)$ et on en déduit la condition initiale $u_0(x) = u_{\text{ex}}(0,x)$, le second membre $f(t,x) = \frac{\partial u_{\text{ex}}}{\partial t}(t,x) - \nu(x) \frac{\partial^2 u_{\text{ex}}}{\partial x^2}(t,x)$ et les conditions aux bords $g(t,0) = u_{\text{ex}}(t,0)$, $g(t,1) = u_{\text{ex}}(t,1)$.

Considère la solution exacte :

$$u_{\text{ex}}(t,x) = \cos(t) \cos(\pi x) \quad (6)$$

Ses dérivées sont :

$$\frac{\partial u_{\text{ex}}}{\partial t} = -\sin(t) \cos(\pi x), \quad (7)$$

$$\frac{\partial^2 u_{\text{ex}}}{\partial x^2} = -\pi^2 \cos(t) \cos(\pi x) \quad (8)$$

On en déduit le second membre :

$$f(t, x) = -\sin(t) \cos(\pi x) + \nu(x) \pi^2 \cos(t) \cos(\pi x) \quad (9)$$

Et les conditions aux bords :

$$g(t, 0) = \cos(t), \quad g(t, 1) = -\cos(t) \quad (10)$$

Voici le code :

```

1 // Conductivité thermique en fonction de x
2 double v(double x){
3     if(x < 0.5){
4         return 1.0; // barre avec conductivité 1
5     }
6     else return 10.0; // barre avec conductivité 10
7 }
8
9 // Solution exacte
10 double uex(double t, double x) {
11     return cos(t) * cos(M_PI*x);
12 }
13
14 // Condition initiale
15 double u0(double x) {
16     return uex(0.0, x);
17 }
18
19 // Second membre de l'équation
20 double f(double t, double x) {
21     double v_val = v(x);
22     return -sin(t) * cos(M_PI * x)
23         + M_PI * M_PI * v_val * cos(t) * cos(M_PI * x);
24 }
25
26 // Conditions aux bords
27 double g0(double t) {
28     return cos(t) * cos(0.0);
29 }
30
31 double g1(double t) {
32     return cos(t) * cos(M_PI*1.0);
33 }

```

Explications des fonctions :

- $v(x)$: définit la conductivité thermique selon la position x , ici deux matériaux.
- $u_{\text{ex}}(t, x)$: solution exacte $u_{\text{ex}}(t, x) = \cos(t) \cos(\pi x)$.
- $u_0(x)$: condition initiale $u(0, x) = u_{\text{ex}}(0, x)$.
- $f(t, x)$: second membre $f(t, x) = \frac{\partial u_{\text{ex}}}{\partial t} - \nu(x) \frac{\partial^2 u_{\text{ex}}}{\partial x^2}$.
- $g_0(t)$, $g_1(t)$: conditions aux bords $u(t, 0)$ et $u(t, 1)$.

3.2 Chaleur explicite, Chaleur implicite

On crée la fonction ν , le temps final T , le nombre de mailles N , et le vecteur correspondant à la discrétisation de la solution initiale u_0 , écrire respectivement les fonctions nommées `chaleur_explicite` et `chaleur_implicite` qui implémentent les méthodes respectives.

L'argument de sortie de ces fonctions sera le vecteur correspondant à l'approximation de la solution au temps $T = Kk$ (noté U^K).

Remarque :

- la fonction `prod_mat_vect` développée au TP3 permet de calculer une itération en temps du schéma (Sexpl) ;
- les fonctions `factorisation_lu` et `resolution_lu` développées au TP3 permettent de résoudre le système linéaire dans le schéma (Simpl).

Question : Pourquoi ici on ne peut pas utiliser la factorisation de Crout LDL^T ?

⇒ On ne peut pas utiliser la factorisation de Crout LDL^T car la matrice associée au schéma implicite n'est pas nécessairement symétrique ni définie positive, contrairement au problème de Poisson 1D.

En fait, La factorisation LDL^T (Crout- LDL^T) est une factorisation pour matrices symétriques (réelles) — en fait, pour matrices symétriques (ou hermitiennes), on peut écrire

$$A = LDL^T$$

si A est symétrique définie (ou parfois avec permutations). Si la matrice n'est pas symétrique, la factorisation LDL^T n'est pas applicable (ou du moins pas directement) : la décomposition n'existe pas en général et la procédure numérique supposant la symétrie échoue. Dans notre discrétisation directe de $\nu(x) \partial_{xx} u$ en évaluant ν en x_i (i.e.

$$\nu_i \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2},$$

) la matrice tri-diagonale du terme spatial a pour coefficients hors diagonale proportionnels à ν_i (la ligne i connecte $i \pm 1$ via ν_i). Ainsi, l'élément $A_{i,i+1}$ n'est en général pas égal à $A_{i+1,i}$ (sauf si $\nu_i = \nu_{i+1}$). La matrice est donc non symétrique alors la factorisation LDL^T est interdite.

Code : Voici l'implémentation des schémas explicite et implicite :

mat_bande : Structure représentant une matrice tridiagonale avec les indices des diagonales et les valeurs stockées dans un tableau 2D.

```
1 using namespace std;
2
3 // Structure pour une matrice bande
4 struct mat_bande{
5     int n; // taille de la matrice
6     vector<int> IND; // indices relatifs des diagonales
7     vector<vector<double>> TAB; // valeurs des diagonales
8 };
```

Listing 3 – Définition des structures et fonctions pour le problème de la chaleur 1D

matrice_1D_bande : Génère une matrice tridiagonale de taille N avec c_0 sur la diagonale principale et c_1 sur les sous- et super-diagonales.

```

1 // Cr ation d'une matrice tridiagonale 1D
2 mat_bande matrice_1D_bande(int N, double c0, double c1) {
3     mat_bande A;
4     A.n = N;
5     int d = 3;
6     A.IND.resize(d);
7     A.TAB.resize(N, vector<double>(d,0.0));
8
9     for(int k=0;k<d;k++)
10         A.IND[k] = k - (d-1)/2; // -1,0,1
11
12     for(int i=0;i<N;i++){
13         for(int k=0;k<d;k++){
14             int j = i + A.IND[k];
15             if(j>=0 && j<N){
16                 if(k==0) A.TAB[i][k] = c1; // sub-diagonal
17                 else if(k==1) A.TAB[i][k] = c0; // diagonal
18                 else if(k==2) A.TAB[i][k] = c1; // super-diagonal
19             }
20         }
21     }
22
23     return A;
24 }

```

prod_mat_vect : Calcule $y = Ax$ pour une matrice bande A et un vecteur x .

```

1 // Multiplication matrice bande par vecteur
2 template <typename T>
3 vector<T> prod_mat_vect(const mat_bande& A, const vector<T>& x){
4     int nd = A.IND.size();
5     int n = A.TAB.size();
6     vector<T> y(n,0.0);
7     for(int i=0;i<n;i++){
8         for(int k=0;k<nd;k++){
9             int j = i + A.IND[k];
10             if(j>=0 && j<n) y[i] += A.TAB[i][k]*x[j];
11         }
12     }
13     return y;
14 }

```

factorisation_LU_tridiag : Prépare la matrice pour résoudre $MU = B$ dans le schéma implicite.

resoudre_LU_tridiag : Résout le système triangulaire supérieur et inférieur pour obtenir U .

```

1 void factorisation_LU_tridiag(mat_bande& A){
2     int n = A.TAB.size();
3
4     A.TAB[0][1] = A.TAB[0][1]; // beta_1
5     A.TAB[0][2] = A.TAB[0][2]; // gamma_1
6     if(n>1) A.TAB[1][0] = A.TAB[1][0] / A.TAB[0][1]; // alpha_2
7

```

```

8   for(int i=1; i<n; i++){
9       double gamma_prev = A.TAB[i-1][2];           // gamma_{i-1} = c_{i-1}
10      double alpha_i = A.TAB[i][0];                 // alpha_i = A.TAB[i][0]
11      A.TAB[i][1] = A.TAB[i][1] - alpha_i * gamma_prev; // beta_i = b_i - alpha_i * gamma_{i-1}
12      if(i < n-1)                                     // alpha_{i+1} = a_{i+1} / beta_i
13          A.TAB[i+1][0] = A.TAB[i+1][0] / A.TAB[i][1];
14      A.TAB[i][2] = A.TAB[i][2];                     // gamma_i = c_i
15  }
16 }
17
18 template <typename T>
19 vector<T> resoudre_LU_tridiag(const mat_bande& A, const vector<T>& b){
20     int n = A.TAB.size();
21     vector<T> x = b;
22     for(int i=1; i<n; i++){
23         x[i] -= A.TAB[i][0] * x[i-1];
24     }
25     x[n-1] /= A.TAB[n-1][1];
26     for(int i=n-2; i>=0; i--){
27         x[i] = (x[i] - A.TAB[i][2] * x[i+1]) / A.TAB[i][1];
28     }
29     return x;
30 }

```

La fonction $v(x)$ est définie par :

```

1 double v(double x){
2     if(x < 0.5){
3         return 1.0;
4     }
5     else return 10.0;
6 }

```

Explications du code :

- La fonction $v(x)$ définit la conductivité thermique $\nu(x)$ selon la position x .
- On a $\nu(x) = 1$ pour $x < 0.5$ et $\nu(x) = 10$ pour $x \geq 0.5$, ce qui modélise une barre composée de deux matériaux de conductivités différentes.
- Cette fonction est utilisée dans le calcul des coefficients de diffusion dans les schémas numérique explicite et implicite.

Schéma implicite (Simpl) Le schéma implicite (Simpl) s'écrit sous la forme :

$$\begin{cases} \frac{u_i^{n+1} - u_i^n}{k} - \frac{\nu}{h^2} (u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}) = f(t_{n+1}, x_i), & \forall 1 \leq i \leq N, \forall n \geq 0, \\ u_i^0 = u_0(x_i), & \forall 1 \leq i \leq N, \\ u_0^{n+1} = 0 = u_{N+1}^{n+1}, & \forall n \geq 0. \end{cases}$$

Simpl s'appuie sur le schéma d'Euler implicite (pour $Y_0 = F(t, Y)$). Le vecteur $(u_i^n)_{1 \leq i \leq N}$ s'obtient par résolution d'un système linéaire dont le second membre dépend

de $(u_i^{n+1})_{1 \leq i \leq N}$ et de $(f(t_n, x_i))_{1 \leq i \leq N}$. On se retrouve à résoudre à chaque pas de temps le système linéaire :

$$MU^n = B^n,$$

avec

$$M = I + \nu k A, \quad B^n = (b_i^n)_{1 \leq i \leq N} \quad \text{tel que } b_i^n = u_i^{n-1} + k f(t^n, x_i),$$

et

$$A = \frac{1}{h^2} \begin{pmatrix} 2 & -1 & 0 & \cdots & 0 & 0 \\ -1 & 2 & -1 & \ddots & & \vdots \\ 0 & \ddots & \ddots & \ddots & & 0 \\ \vdots & & \ddots & 2 & -1 & 0 \\ 0 & \cdots & 0 & -1 & 2 & -1 \\ 0 & \cdots & 0 & 0 & -1 & 2 \end{pmatrix}.$$

À chaque pas de temps n , cette résolution permet de calculer $U^n = (u_i^n)_{1 \leq i \leq N}$.

Remarque : Le schéma (**Simpl**) est **stable** sans aucune condition sur les pas k et h donc on prendra $k = h$ par exemple.

```

1 template <typename T>
2 vector< T> chaleur_implicite(const mat_bande&A, int N, double h, const
   vector<T>& nu, double Tfinal, const vector<T>& u0) {
3     double k = h;
4     int K = static_cast<int>(Tfinal / k);
5
6     // Initialisation
7     vector<T> U = u0;
8
9     // Construction de la matrice bande M = I + kA
10    mat_bande M = A; // copie pour modification
11    for (int i = 0; i < N; ++i) {
12        double alpha = nu[i] * k / (h * h);
13        M.TAB[i][1] = 1.0 + 2.0 * alpha; // diagonale principale
14        if (i > 0) M.TAB[i][0] = -alpha; // sous-diagonale
15        if (i < N-1) M.TAB[i][2] = -alpha; // sur-diagonale
16    }
17
18    M.TAB[0][0] = 0.0; M.TAB[0][1] = 1.0; M.TAB[0][2] = 0.0;
19    M.TAB[N-1][0] = 0.0; M.TAB[N-1][1] = 1.0; M.TAB[N-1][2] = 0.0;
20
21    mat_bande M_copy = M; // copie pour factorisation
22    factorisation_LU_tridiag(M_copy);
23
24    for (int n = 0; n < K; ++n) {
25        vector<T> B(N);
26        double t_next = (n+1) * k;
27        for (int i = 0; i < N; ++i) {
28            double xi = (i + 1) * h;
29            B[i] = U[i] + k * f(t_next, xi); // f(t^{n+1}, x_i)
30        }
31        B[0] = g0(t_next); // Conditions aux bords
32        B[N-1] = g1(t_next);
33        U = resoudre_LU_tridiag(M_copy, B);
34        U[0] = g0(t_next); // impose de nouveau les conditions aux
   bords

```

```

35     U[N-1] = g1(t_next);
36 }
37
38 return U; // approximation au temps final T = K * k
39 }

```

Cette implémentation s'appuie sur la même idée que la construction de la matrice profil présentée précédemment. La résolution du schéma implicite conduit à résoudre à chaque pas de temps le système linéaire

$$MU^n = B^n, \quad \text{avec } M = I + \nu k A.$$

Ici, on construit A sous forme de matrice bande (storage bande) :

$$\text{ind} = (-1, 0, 1), \quad \text{ATAB} = \frac{1}{h^2} \begin{pmatrix} * & 2 & -1 \\ -1 & 2 & -1 \\ \vdots & \ddots & \vdots \\ -1 & 2 & -1 \\ -1 & 2 & * \end{pmatrix}.$$

Ainsi, chaque itération temporelle correspond à une résolution linéaire en utilisant cette matrice bande, de manière analogue à ce qui est fait pour la matrice profil.

Explications du code :

- Initialisation : Le vecteur U est initialisé avec la condition initiale u_0 . Il contiendra l'approximation de la solution à chaque pas de temps.
- Construction de la matrice bande M :
 - On part de la matrice de diffusion tridiagonale A (stockée en format bande).
 - Pour chaque point i , on calcule $\alpha = \nu_i k / h^2$.
 - La diagonale principale de M devient $1 + 2\alpha$, la sous-diagonale et la sur-diagonale deviennent $-\alpha$.
 - Ceci correspond à la discrétisation implicite du Laplacien avec le schéma d'Euler implicite.
- Conditions aux bords :
 - Les deux premières et dernières lignes de M sont modifiées pour imposer les conditions de Dirichlet : $U_0 = g_0(t)$ et $U_{N-1} = g_1(t)$.
- Factorisation LU tridiagonale :
 - On applique la factorisation LU tridiagonale sur M pour pouvoir résoudre efficacement le système linéaire $MU^{n+1} = B^n$ à chaque pas de temps.
 - Cette factorisation exploite le stockage bande pour réduire la complexité en temps et en mémoire.
- Boucle temporelle : Pour chaque pas n allant de 0 à $K - 1$:
 - On construit le second membre B^n : $B_i^n = U_i^n + k f(t_{n+1}, x_i)$.
 - On impose les conditions aux bords sur B^n .
 - On résout le système linéaire $MU^{n+1} = B^n$ en utilisant la factorisation LU.
 - On ré-impose les conditions aux bords sur U^{n+1} après la résolution.
- Sortie : Le vecteur U retourné correspond à l'approximation de la solution au temps final $T = K \cdot k$.

Schéma explicite (Sexpl) Le schéma (**Sexpl**) repose sur le schéma d'Euler explicite appliqué à l'équation de la chaleur. Il consiste à évaluer le terme source $f(t_n, x_i)$ et les valeurs u_i^n connues au temps t_n pour calculer directement u_i^{n+1} au temps t_{n+1} .

$$\begin{cases} \frac{u_i^{n+1} - u_i^n}{k} - \frac{\nu}{h^2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n) = f(t_n, x_i), & \forall 1 \leq i \leq N, \forall n \geq 0, \\ u_i^0 = u_0(x_i), & \forall 1 \leq i \leq N, \\ u_0^{n+1} = 0 = u_{N+1}^{n+1}, & \forall n \geq 0. \end{cases}$$

En pratique, les valeurs u_i^{n+1} sont obtenues de manière explicite à partir de u_i^n et f selon la formule :

$$u_i^{n+1} = u_i^n + \nu \frac{k}{h^2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n) + k f(t_n, x_i), \quad \forall 1 \leq i \leq N.$$

Remarque : le schéma explicite (**Sexpl**) est conditionnellement stable. Il est stable si et seulement si :

$$k < \frac{h^2}{2 \|\nu\|_\infty}.$$

Le code suivant implémente le schéma explicite décrit précédemment.

```

1 template <typename T>
2 vector<T> chaleur_explicite(const mat_bande& A, int N, double h,
3                             const vector<T>& nu, double Tfinal,
4                             const vector<T>& u0) {
5     double k = (h*h) / (2.0 * (*max_element(nu.begin(), nu.end()))); //
    pas de temps stable
6     int K = static_cast<int>(Tfinal / k);
7     vector<T> U = u0;
8
9     for (int n = 0; n < K; n++) {
10         double t_n = n * k;
11         vector<T> Fn(N);
12         for (int i = 0; i < N; i++)
13             Fn[i] = f(t_n, (i + 1) * h);
14
15         // Construction de la matrice L = (nu * k / h^2) * A
16         mat_bande L = A;
17         for (int i = 0; i < N; i++) {
18             L.TAB[i][1] = 2.0 * nu[i] * k / (h * h);
19             if (i > 0)
20                 L.TAB[i][0] = -nu[i] * k / (h * h);
21             if (i < N - 1)
22                 L.TAB[i][2] = -nu[i] * k / (h * h);
23         }
24
25         // Calcul du produit L * U^n
26         vector<T> LU = prod_mat_vect(L, U);
27
28         // Mise à jour explicite : U^{n+1} = U^n - (L * U^n) + k * F^n
29         for (int i = 0; i < N; i++)
30             U[i] = U[i] - LU[i] + k * Fn[i];
31
32         // Conditions aux bords
33         U[0] = g0(t_n + k);
34         U[N - 1] = g1(t_n + k);

```

```

35     }
36
37     return U;
38 }

```

Listing 4 – Implémentation du schéma explicite (Sexpl)

Explications du code :

- La fonction `chaleur_explicite` résout l'équation de la chaleur à une dimension en utilisant le schéma d'Euler explicite (Sexpl). Ce schéma est conditionnellement stable, c'est-à-dire qu'il impose une contrainte sur le pas de temps k et le pas d'espace h pour garantir la stabilité numérique.
- Le pas de temps k est choisi selon la condition de stabilité du schéma explicite :

$$k \leq \frac{h^2}{2 \max_i \nu_i},$$

ce qui assure que les oscillations numériques ne se propagent pas et que la solution reste stable. Dans le code :

```

1     double k = (h*h) / (2.0 * (*max_element(nu.begin(), nu.end())))
2         );

```

la fonction `max_element` cherche le maximum de ν_i , et la formule impose la borne supérieure pour k .

- Le nombre total de pas de temps K est ensuite calculé par :

```

1     int K = static_cast<int>(Tfinal / k);
2

```

Cette ligne convertit le rapport T_{final}/k en entier, indiquant combien d'itérations temporelles sont nécessaires pour atteindre le temps final T_{final} . En pratique, cela signifie que la boucle temporelle `for (n < K)` effectuera K itérations, chacune correspondant à une évolution de la solution de t_n à $t_{n+1} = t_n + k$.

- Le vecteur U est initialisé par la condition initiale u_0 :

$$U_i^0 = u_0(x_i), \quad 1 \leq i \leq N.$$

- Dans la boucle temporelle principale `for (n < K)`, le programme avance la solution dans le temps :

1. On calcule le vecteur source $F^n = f(t_n, x_i)$, qui correspond au terme de forçage du problème.
2. On construit la matrice tridiagonale L représentant l'opérateur discret du Laplacien :

$$L = \frac{\nu k}{h^2} \begin{pmatrix} * & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ 0 & -1 & 2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -1 \\ 0 & \cdots & 0 & -1 & * \end{pmatrix}.$$

Dans le code :

```

1     L.TAB[i][1] = 2.0 * nu[i] * k / (h*h);
2     if (i > 0)    L.TAB[i][0] = -nu[i] * k / (h*h);
3     if (i < N-1) L.TAB[i][2] = -nu[i] * k / (h*h);
4

```

cette matrice est stockée sous forme *bande* afin de limiter la mémoire utilisée et d'accélérer les calculs.

3. On calcule ensuite le produit matriciel $LU^n = L \times U^n$, puis on met à jour la solution selon la formule explicite :

$$U^{n+1} = U^n - LU^n + kF^n.$$

Cela correspond à la discrétisation temporelle du type d'Euler explicite appliquée à l'équation de la chaleur.

4. Les conditions aux bords de Dirichlet sont imposées à chaque pas de temps :

$$U_0^{n+1} = g_0(t_{n+1}), \quad U_{N-1}^{n+1} = g_1(t_{n+1}).$$

Dans le code :

```
1      U[0]      = g0(t_n + k);
2      U[N-1]   = g1(t_n + k);
3
```

cela garantit que la solution numérique reste cohérente avec les conditions physiques imposées sur les bords.

- Après la dernière itération, la fonction renvoie le vecteur U représentant l'approximation de la solution au temps final $T = K \cdot k$.
- En résumé :
 - Le schéma est simple à implémenter et rapide à exécuter.
 - Sa stabilité dépend fortement du choix du pas de temps k .
 - Il est souvent utilisé pour valider des schémas plus complexes (comme le schéma implicite) car il fournit une base de comparaison intuitive.

3.3 Vérification de chaleur

On écrit respectivement les fonctions `verif_chaleur_explicite(T, N)` et `verif_chaleur_implicite(T, N)` qui résolvent l'équation de la chaleur avec les données T, N , la fonction ν donnée dans (3) et les fonctions u_0, f et g calculées précédemment à partir de u_{ex} . Elles écrivent N et l'erreur finale $\varepsilon_N = \max_j |u_j^K - u_{ex}(T, x_j)|$ à l'écran et dans un fichier "file".

Vérification de Simpl

```
1 // Fonction de v r i f i c a t i o n d e l a m e t h o d e i m p l i c i t e
2 void verif_chaleur_implicite(double Tfinal, int N, const string& file) {
3     double h = 1.0 / (N + 1);
4     double c0 = 2.0/(h*h);
5     double c1 = -1.0/(h*h);
6
7     vector<double> u0_vec(N);
8     vector<double> nu_vec(N);
9     for(int i=0;i<N;i++){
10         double xi = i*h;
11         u0_vec[i] = u0(xi);
12         nu_vec[i] = v(xi);
13     }
14
15     mat_bande A = matrice_1D_bande(N, c0, c1);
```

```

16     vector<double> U = chaleur_implicite(A, N, h, nu_vec, Tfinal, u0_vec
    ); // U(T)
17
18     vector<double> Uex(N);
19     double errmax = 0.0;
20     for (int j = 0; j < N; ++j) {
21         double xj = (j + 1) * h;
22         Uex[j] = uex(Tfinal, xj);
23         errmax = max(errmax, fabs(U[j] - Uex[j]));
24     }
25     cout << " Erreur maximale = " << errmax << endl;
26
27     //  criture  dans un fichier
28     ofstream fout(file, ios::app); // append
29     if (fout.is_open()) {
30         fout << N << " \t " << errmax << endl;
31         fout.close();
32     } else {
33         cerr << "Erreur: impossible d'ouvrir le fichier " << file <<
endl;
34     }
35 }

```

Explications du code :

- Calcul du pas spatial $h = 1/(N + 1)$ et coefficients $c0$ et $c1$ pour la matrice tridiagonale.
- Initialisation des vecteurs u_0 et ν à partir des fonctions définies précédemment.
- Construction de la matrice bande A pour le Laplacien discret.
- Appel de la fonction `chaleur_implicite` pour obtenir l'approximation U au temps final T_{final} .
- Calcul de l'erreur maximale $\varepsilon_N = \max_j |U_j - u_{\text{ex}}(T, x_j)|$.
- Affichage de l'erreur et écriture dans un fichier pour sauvegarde.

Vérification de Sexpl

```

1 // Fonction de v rification de la m thode explicite
2 void verif_chaleur_explicite(double Tfinal, int N, const string& file) {
3     double h = 1.0 / (N + 1);
4     double c0 = 2.0/(h*h);
5     double c1 = -1.0/(h*h);
6
7     vector<double> u0_vec(N);
8     vector<double> nu_vec(N);
9     for(int i=0;i<N;i++){
10         double xi = i*h;
11         u0_vec[i] = u0(xi);
12         nu_vec[i] = v(xi);
13     }
14
15     mat_bande A = matrice_1D_bande(N, c0, c1);
16     vector<double> U = chaleur_explicite(A, N, h, nu_vec, Tfinal, u0_vec
    ); // U(T)
17
18     vector<double> Uex(N);
19     double errmax = 0.0;
20     for (int j = 0; j < N; ++j) {
21         double xj = (j + 1) * h;

```

```

22     Uex[j] = uex(Tfinal, xj);
23     errmax = max(errmax, fabs(U[j] - Uex[j]));
24 }
25 cout << " Erreur maximale = " << errmax << endl;
26
27 //  criture  dans un fichier
28 ofstream fout(file, ios::app); // append
29 if (fout.is_open()) {
30     fout << N << " \t " << errmax << endl;
31     fout.close();
32 } else {
33     cerr << "Erreur: impossible d'ouvrir le fichier " << file <<
endl;
34 }
35 }

```

Explications du code :

- Calcul du pas spatial $h = 1/(N + 1)$ et coefficients $c0$ et $c1$ pour la matrice tridagonale.
- Initialisation des vecteurs u_0 et ν à partir des fonctions définies précédemment.
- Construction de la matrice bande A pour le Laplacien discret.
- Appel de la fonction `chaleur_explicite` pour obtenir l'approximation U au temps final T_{final} .
- Calcul de l'erreur maximale $\varepsilon_N = \max_j |U_j - u_{\text{ex}}(T, x_j)|$.
- Affichage de l'erreur et écriture dans un fichier pour sauvegarde.

3.4 Tests

Grâce à un `switch/case`, le programme principal exécutera soit `verif_chaleur_explicite` soit `verif_chaleur_implicit` pour $T = 1.0$, en prenant $N = \{4, 8, 16, 32, 64, 128, \dots\}$. Sauver les résultats dans un fichier, où dans chaque ligne on écrira N, ε_N .

```

1 // Programme principal pour tester les m thodes explicite et implicite
2 int main() {
3     double Tfinal = 1.0;
4     string fichier = "erreur2.txt";
5
6     // menu pour choisir la m thode
7     int choix;
8     cout << "Choisir la m thode:\n";
9     cout << "1 - Euler explicite\n";
10    cout << "2 - Euler implicite\n";
11    cout << "Votre choix: ";
12    cin >> choix;
13
14    // Initialisation du fichier de sortie
15    ofstream fout(fichier);
16    if(fout.is_open()){
17        fout << "N \t Erreur max \n";
18        fout.close();
19    }
20
21    vector<int> Ns = {4, 8, 16, 32, 64, 128, 256};
22    switch(choix){
23        case 1: // Euler explicite
24            for(int N : Ns){

```

```

25         cout << "R solution explicite pour N = " << N << endl;
26         verif_chaleur_explicite(Tfinal, N, fichier);
27     }
28     break;
29     case 2: // Euler implicite
30     for(int N : Ns){
31         cout << "R solution implicite pour N = " << N << endl;
32         verif_chaleur_implicite(Tfinal, N, fichier);
33     }
34     break;
35     default:
36         cerr << "Choix invalide !" << endl;
37         return 1;
38     }
39     cout << "\nR sultats sauvegard s dans : " << fichier << endl;
40     return 0;
41 }

```

Explications du code :

- Le programme principal demande à l'utilisateur de choisir la méthode à tester : Euler explicite ou implicite.
- Il initialise le fichier de sortie pour sauvegarder N et l'erreur maximale.
- La liste des mailles N est définie par $\{4, 8, 16, 32, 64, 128, 256\}$.
- Selon le choix de l'utilisateur (switch/ case) :
 - Pour Euler explicite, on appelle `verif_chaleur_explicite` pour chaque N .
 - Pour Euler implicite, on appelle `verif_chaleur_implicite` pour chaque N .
- Les résultats sont affichés à l'écran et sauvegardés dans le fichier `erreur2.txt`.
- Lorsque le programme est lancé, il affiche le menu de sélection de la méthode :
 Choisir la méthode:
 1 - Euler explicite \\ L'utilisateur doit entrer 1 ou 2 pour choisir la méthode
 2 - Euler implicite
 Votre choix:

Test de Sexpl : Lors de l'exécution du programme avec la méthode Euler explicite, on obtient par exemple les résultats suivants :

```

1 R solution explicite pour N = 4
2 Erreur maximale = 0.453176
3 R solution explicite pour N = 8
4 Erreur maximale = 0.191776
5 R solution explicite pour N = 16
6 Erreur maximale = 0.0614132
7 R solution explicite pour N = 32
8 Erreur maximale = 0.0173485
9 R solution explicite pour N = 64
10 Erreur maximale = 0.0046084
11 R sultats sauvegard s dans : erreur2.txt

```

Ces valeurs montrent que l'erreur diminue lorsque N augmente, ce qui illustre la convergence de la méthode explicite.

Test de Simp : Lors de l'exécution du programme avec la méthode Euler implicite, on obtient par exemple les résultats suivants :

```

1 R solution implicite pour N = 4
2 Erreur maximale = 0.449299
3 R solution implicite pour N = 8
4 Erreur maximale = 0.191546
5 R solution implicite pour N = 16
6 Erreur maximale = 0.061615
7 R solution implicite pour N = 32
8 Erreur maximale = 0.0175243
9 R solution implicite pour N = 64
10 Erreur maximale = 0.00471508
11 R sultats sauvegard s dans : erreur2.txt

```

On constate que l'erreur diminue également lorsque N augmente, illustrant la convergence de la méthode implicite. Comparativement à la méthode explicite, la méthode implicite reste stable et permet des pas de temps plus grands sans condition de stabilité. L'Euler explicite nécessite de respecter la condition de stabilité $k < h^2/(2\nu_\infty)$.

Ces observations confirment la validité des schémas numériques mis en œuvre et illustrent les propriétés théoriques de convergence et de stabilité des méthodes explicite et implicite pour l'équation de la chaleur.

3.5 Graphe

On réutilise le fichier de code Python précédent pour tracer le nuage de points $(\log(N), \log(\varepsilon_N))$ et calculer la droite de régression afin de déterminer expérimentalement le taux de convergence β dans $\varepsilon_N \approx C h_N^\beta$.

Graphe de Sexpl :

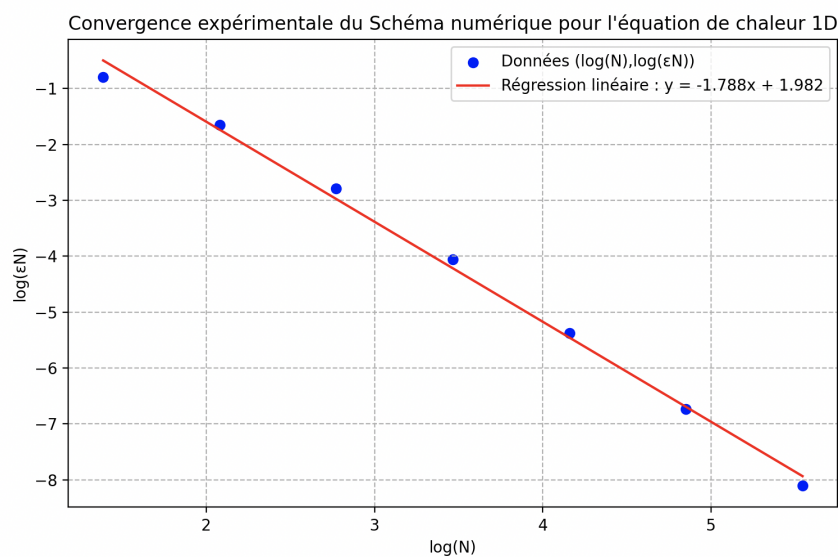


FIGURE 3 – Nuage de points $(\log(N), \log(\varepsilon_N))$ avec droite de régression.

Taux de convergence expérimental : $\beta \approx 1.738$, Constante $C \approx 6.391 \times 10^0$.

Graphe de Simpl :

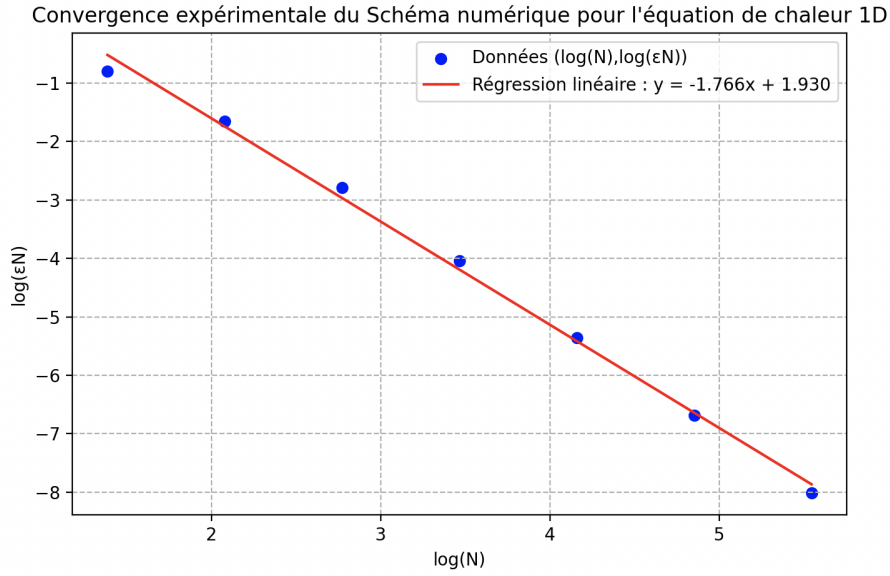


FIGURE 4 – Nuage de points $(\log(N), \log(\varepsilon_N))$ avec droite de régression.

Taux de convergence expérimental : $\beta \approx 1.766$, Constante $C \approx 6.888 \times 10^0$.

Observation

Les résultats numériques montrent que l'erreur maximale diminue lorsque le nombre de mailles N augmente pour les deux schémas, illustrant la convergence vers la solution exacte $u_{\text{ex}}(t, x)$.

À partir de la régression log-log des erreurs, on obtient un taux de convergence expérimental $\beta \approx 1.738$ et une constante $C \approx 6.391$ pour Euler explicite, et $\beta \approx 1.766$ et $C \approx 6.888$ pour Euler implicite. Ces valeurs sont légèrement inférieures à l'ordre théorique 2 en espace, ce qui peut s'expliquer par l'utilisation d'Euler explicite et implicite (ordre 1 en temps) et par la discontinuité de la conductivité $\nu(x)$.

On remarque que les deux méthodes fournissent des erreurs finales très proches, mais Euler implicite présente un avantage en termes de stabilité, permettant d'utiliser des pas de temps plus grands sans risque d'instabilité.

4 Résolution de l'équation de Black-Scholes

On cherche $u \in C^{1,2}([0, T] \times (0, L))$ solution de l'équation de Black-Scholes réécrite comme une équation de la chaleur avec donnée initiale (en ayant changé la variable t en $T - t$) :

$$\begin{cases} \frac{\partial u(t, x)}{\partial t} - \frac{1}{2}\sigma^2 x^2 \frac{\partial^2 u(t, x)}{\partial x^2} = 0, & \forall (t, x) \in (0, T] \times (0, L), \\ u(0, x) = h(x), & \forall x \in (0, L), \\ u(t, 0) = 0, & \forall t \in [0, T], \quad (\text{C.L. de Dirichlet}), \\ \frac{\partial u(t, L)}{\partial x} = 0, & \forall t \in [0, T], \quad (\text{C.L. de Neumann}). \end{cases}$$

Dans le cas d'un call européen (option d'achat), on fixera par exemple le strike de l'option à $K(x) = 0.95x$ avec $x \in (0, L)$ le cours actuel de l'action. On rappelle que dans ce cas $h(x) = \max(x - K, 0)$. Si l'on note h le pas d'espace et k le pas de temps, on utilisera le schéma implicite (Simpl) qui est toujours stable, donc on prendra par exemple $k = h$.

4.1 Initialisation

On crée le vecteur X de discrétisation de l'intervalle $[0, L]$ avec N sous-intervalles (donc $N + 1$ points si l'on compte les deux extrémités), définir les fonctions $K(x)$ et $u_0(L, N)$ pour le payoff du contrat d'achat.

```

1 // on crée vecteur X de taille M = N+1 car on inclut les deux bornes 0
  et L
2 vector<double> generer_X(double L, int N) {
3     int M = N+1;
4     vector<double> X(M);
5     double h = L / double(N); // note: h = L/N
6     for(int i=0; i<M; i++) X[i] = i * h;
7     return X;
8 }
9
10 // strike K(x)
11 double K_of_x(double x) { return 0.95 * x; }
12
13 // payoff initial h(x) = max(x - K(x), 0)
14 vector<double> u0_call(const vector<double>& X) {
15     int M = X.size();
16     vector<double> u0(M);
17     for(int i=0; i<M; i++){
18         u0[i] = max(X[i] - K_of_x(X[i]), 0.0);
19     }
20     return u0;
21 }

```

Listing 5 – Génération du vecteur X et condition initiale pour l'option call

Explications du code :

- La fonction `generer_X(double L, int N)` crée le vecteur des points $X_i = i \times h$ avec $h = L/N$, en incluant les deux bornes 0 et L .
- La fonction `K_of_x(double x)` définit le strike dépendant du prix x , ici $K(x) = 0.95x$.
- La fonction `u0_call(const vector<double>& X)` calcule la condition initiale du problème de Black-Scholes correspondant au payoff d'une option call européenne :

$$h(x) = \max(x - K(x), 0).$$

4.2 Matrice bande pour Black-Scholes

On crée le vecteur X , la volatilité σ et les pas h et k , définir la fonction `matrice_BS_bande` qui assemble la matrice tridiagonale de Black-Scholes en stockage bande. Attention à la condition de Neumann en $x = L$.

Remarque : On va également réutiliser les structures `mat_bande` et la fonction `matrice_1D_bande` du module `Simpl` de la section précédente.

```

1 mat_bande matrice_BS_bande(const vector<double>& X, double h, double k,
  double sigma){
2     int M = X.size();
3     mat_bande A;
4     A.n = M;
5     A.IND = { -1, 0, 1 };
6     A.TAB.assign(M, vector<double>(3, 0.0));
7
8     for(int i=0;i<M;i++){
9         double nu = 0.5 * sigma * sigma * X[i] * X[i]; // nu_i
10        double factor = k * nu / (h*h);
11        if(i==0){
12            // Dirichlet sur x=0: u=0
13            A.TAB[i][0] = 0.0;
14            A.TAB[i][1] = 1.0;
15            A.TAB[i][2] = 0.0;
16        } else if(i == M-1){
17            // Neumann sur x=L: du/dx=0 => u_{M-1} = u_{M-2}
18            // on utilise une condition fantome pour eliminer u_{M}
19            A.TAB[i][0] = -2.0 * factor;
20            A.TAB[i][1] = 1.0 + 2.0 * factor;
21            A.TAB[i][2] = 0.0;
22        } else {
23            // standard interior
24            A.TAB[i][0] = -factor;
25            A.TAB[i][1] = 1.0 + 2.0*factor;
26            A.TAB[i][2] = -factor;
27        }
28    }
29    return A;
30 }

```

Listing 6 – Construction de la matrice bande pour le schéma implicite Black–Scholes

Explications du code :

- La fonction `matrice_BS_bande(const vector<double>& X, double h, double k, double sigma)` assemble la matrice tridiagonale en stockage bande pour le schéma implicite Black–Scholes.
- $M = X.size()$: nombre de points sur la grille.
- $factor = k \nu_i / h^2$ avec $\nu_i = 1/2 \sigma^2 X_i^2$.
- Pour $i = 0$: condition de Dirichlet $u = 0$ en $x = 0$.
- Pour $i = M - 1$: condition de Neumann $\frac{du}{dx} = 0$ en $x = L$, implémentée avec un point fantôme.
- Pour les points intérieurs : standard tridiagonal avec coefficients a_i, b_i, c_i .

4.3 Méthode Simpl pour Black-Sholes

On crée la longueur de l'intervalle L , le temps final T , la volatilité σ , le nombre de mailles N , et le vecteur correspondant à la discrétisation de la solution initiale u_0 , écrire la fonction `BS_implicite` qui implémente la méthode (Simpl). L'argument de sortie de cette fonction sera le vecteur correspondant à l'approximation de la solution au temps T . De plus, à chaque itération en temps n , la solution approchée U^n sera écrite dans un fichier, afin de faire un plot 3D en post-traitement. On remarque que les fonctions `factorisation_lu` et `resolution_lu` développées au TP3 permettent de résoudre le système linéaire.

```

1 // Fonction BS implicite en C++
2 vector<double> BS_implicite(const vector<double>& X, double L, double T,
   double sigma, int N, double h, double k, const vector<double>& u0,
   const string& file_out){
3     int M = X.size();
4     assert(M == N+1);
5     vector<double> U = u0;
6
7     mat_bande Mmat = matrice_BS_bande(X, h, k, sigma);
8     mat_bande Mcopy = Mmat;
9     factorisation_LU_tridiag(Mcopy);
10
11     int Ktime = static_cast<int>(round(T / k));
12
13     ofstream fout(file_out, ios::out | ios::trunc);
14     if(!fout.is_open()){
15         cerr << "Impossible d'ouvrir " << file_out << endl;
16         return U;
17     }
18     fout << "# t ";
19
20
21     for(int i=0;i<M;i++) fout << " x_" << i;
22     fout << "\n";
23     for(int n=0;n<=Ktime;n++){
24         double t = n * k;
25         fout << t;
26         for(int i=0;i<M;i++) fout << " " << U[i];
27         fout << "\n";
28         if(n < Ktime){
29             vector<double> B = U;
30             B[0] = 0.0;
31             U = resoudre_LU_tridiag(Mcopy, B);
32             U[0] = 0.0;
33             U[M-1] = U[M-2];
34         }
35     }
36
37     fout.close();
38     return U;
39 }

```

Explication du code :

- On initialise la solution U avec la condition initiale u_0 .
- La matrice tridiagonale $Mmat$ correspondant à la discrétisation implicite est construite par `matrice_BS_bande`.
- On effectue la factorisation LU de la matrice tridiagonale pour accélérer la résolution du système à chaque pas de temps.
- Le nombre de pas de temps est $Ktime = \text{round}(T/k)$, et on ouvre un fichier `file_out` pour enregistrer les solutions à chaque étape.
- À chaque pas de temps n :
 - On écrit le temps et la solution U^n dans le fichier.
 - Si ce n'est pas le dernier pas, on résout le système linéaire $MU^{n+1} = U^n$ en utilisant la factorisation LU.
 - Les conditions aux frontières sont appliquées : $U_0 = 0$ et $U_{M-1} = U_{M-2}$.
- Le vecteur U final correspondant à $t = T$ est retourné.

4.4 Tests

Écrire le programme principal qui lira les données et exécutera `BS_implicit`. Prendre par exemple : $L = 10000$ (euros), $T = 365$ (jours), $\sigma = 0.2$ et N assez grand pour que la solution calculée soit assez précise. D'autres choix sont aussi possibles.

```
1 // Programme principal
2 int main(){
3     double L = 10000.0;
4     double T = 365.0;
5     double sigma = 0.2;
6     int N = 200;
7     double h = L / double(N+1);
8     double k = h;
9     // cree grid et condition initital
10    vector<double> X = generer_X(L, N);    // size M = N+1
11    vector<double> u0 = u0_call(X);
12
13    string file_out = "BS_implicit.txt";
14    vector<double> Ufinal = BS_implicit(X, L, T, sigma, N, h, k, u0,
15    file_out);
16
17    cout << "Solution calcul e au temps final T = " << T << " (
18    Resultats sauvegardes dans " << file_out << ")\n";
19    cout << "x    U(T,x)\n";
20    for(size_t i=0;i<X.size();i++){
21        cout << X[i] << "    " << Ufinal[i] << "\n";
22    }
23    return 0;
24 }
```

Explication du code :

- On définit les paramètres du problème : L (longueur de l'intervalle), T (temps final), σ (volatilité) et N (nombre de mailles).
- On calcule les pas d'espace $h = L/(N+1)$ et de temps $k = h$ pour la discrétisation.
- La grille spatiale X est générée avec `generer_X(L, N)` et la condition initiale u_0 avec `u0_call(X)`.
- La fonction `BS_implicit` est appelée pour calculer la solution approchée à chaque pas de temps et écrire les résultats dans le fichier `BS_implicit.txt`.
- Enfin, le programme affiche dans la console la solution calculée au temps final T pour chaque point de la grille.

Voici le résultat obtenu après la compilation :

Solution calculée au temps final $T = 365$

(Résultats sauvegardés dans le fichier *BS_implicit.txt*)

x	$U(T, x)$
0	0.00000
50	0.883746
100	1.52403
150	2.05211
200	2.51003
250	2.91807
300	3.28806
350	3.62764
400	3.94213
450	4.23543
500	4.51051
$\vdots$	$\vdots$
9950	13.3514
10000	13.3514

TABLE 1 – Solution numérique $U(T, x)$ au temps final $T = 365$ (méthode implicite).

Remarque :

- x : prix de l'action (variable d'entrée du marché)
- $U(T, x) = v(T, x)$: valeur de l'option (variable de sortie)

$\Rightarrow$ Ce tableau nous indique que : *Si aujourd'hui l'action vaut 10000, alors l'option correspondante vaut environ 13.3514 unités monétaires.*

	Variable	Signification financière
	x	Prix actuel de l'action sur le marché
Explication :	$U(T, x)$	Valeur de l'option à l'échéance ou proche de celle-ci
	$\frac{\partial U}{\partial x}$	Delta : sensibilité du prix de l'option au prix de l'action
	$\frac{\partial^2 U}{\partial x^2}$	Gamma : convexité, mesure du risque non-linéaire

La banque utilise ces informations pour :

- Proposer des prix aux clients ;
- Ajuster le portefeuille de couverture selon l'évolution du marché ;
- Suivre quotidiennement les risques Delta/Gamma.

4.5 Graphe

Avec Python, on va tracer l'évolution de la solution au cours du temps (plot 3D), le payoff du contrat et le prix de l'option d'achat (solution calculée au temps T).

4.5.1 Graphe en 3D

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4
```

```

5 # Chargement des données
6 data = np.loadtxt("BS_implicit.txt", comments="#")
7 T = data[:, 0]          # temps
8 U = data[:, 1:]         # solution numérique
9 X = np.linspace(0, 500, U.shape[1]) # espace
10
11 # Création de la grille X-T
12 X_grid, T_grid = np.meshgrid(X, T)
13
14 # Tracé de la surface 3D
15 fig = plt.figure(figsize=(12,8))
16 ax = fig.add_subplot(111, projection='3d')
17 surf = ax.plot_surface(X_grid, T_grid, U, cmap='hot', edgecolor='none')
18
19 # Colorbar indiquant la valeur de la solution
20 cbar = fig.colorbar(surf, ax=ax, shrink=0.6, aspect=10)
21 cbar.set_label('Valeur de la solution U(X,T)', fontsize=12)
22
23 # Noms des axes
24 ax.set_xlabel('Position X (espace)', fontsize=12)
25 ax.set_ylabel('Temps T', fontsize=12)
26 ax.set_zlabel('Hauteur = Valeur U(X,T)', fontsize=12)
27
28 # Titre
29 ax.set_title(" Évolution du payoff du contrat et le prix de l'option d'achat",
30             fontsize=14)
31
32 # Vue 3D
33 ax.view_init(elev=30, azim=-60)
34 plt.show()

```

Explication du code :

- On importe les bibliothèques nécessaires : `numpy` pour le calcul numérique et `matplotlib` pour le tracé.
- On charge les données calculées par `BS_implicit` depuis le fichier `BS_implicit.txt`. La première colonne correspond au temps T et les colonnes suivantes à la solution numérique $U(X, T)$.
- On construit un vecteur X représentant les positions spatiales, puis une grille X_grid, T_grid pour le tracé 3D avec `meshgrid`.
- La fonction `plot_surface` crée la surface 3D de la solution, avec la colormap `hot` pour visualiser l'intensité.
- On ajoute une barre de couleur (`colorbar`) pour indiquer les valeurs de $U(X, T)$.
- Les axes et le titre sont définis avec `set_xlabel`, `set_ylabel`, `set_zlabel` et `set_title`.
- La vue 3D est ajustée avec `view_init` pour un angle de visualisation agréable.
- Enfin, `plt.show()` affiche la figure.

Graphe en 3D

Évolution du payoff du contrat et le prix de l'option d'achat

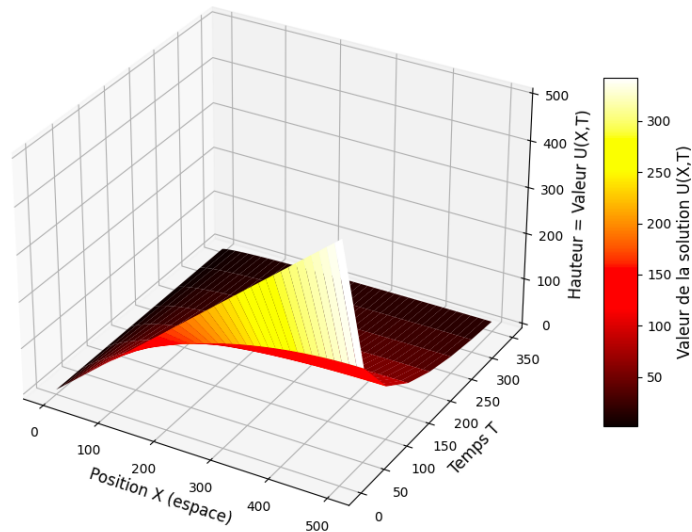


FIGURE 5 – Évolution du payoff du contrat et le prix de l'option d'achat.

Signification de la simulation : Le graphique 3D de $u(t, x)$ montre :

Axe x : prix de l'action, Axe t : temps, Axe u : valeur de l'option.

Il permet à la banque de visualiser comment le prix de l'option évolue dans le temps et de mettre en place une stratégie de couverture (*Delta hedging*) adaptée.

Analyse du graphique : La surface obtenue représente l'évolution de la solution numérique $U(X, T)$ de l'équation de Black–Scholes réécrite sous forme d'une équation de la chaleur. On observe que :

- Au temps initial $T = 0$, la solution correspond bien au *payoff* du call européen, défini par $h(x) = \max(x - K, 0)$.
- Lorsque le temps augmente, la surface devient plus lisse et tend à diminuer, traduisant l'effet de la décote temporelle (*time decay*) du prix de l'option.
- La condition de Dirichlet $u(t, 0) = 0$ est bien respectée, de même que la condition de Neumann $\partial_x u(t, L) = 0$ au bord droit.
- La forme globale de la surface, convexe et décroissante selon T , est cohérente avec le comportement théorique du prix d'un call européen : plus la maturité approche, plus la valeur de l'option tend vers le *payoff* final.

Ces observations confirment la stabilité et la cohérence du schéma implicite utilisé pour la résolution numérique.

4.5.2 Graphe en Heat Map

On peut également utiliser une **heat map** pour visualiser l'évolution de la solution $U(X,T)$ de manière intuitive : la couleur indique directement la valeur de la solution, ce qui permet de repérer rapidement les zones de forte ou faible valeur et d'observer l'évolution dans le temps et l'espace.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Chargement des données
5 data = np.loadtxt("BS_implicit.txt", comments="#")
6 T = data[:, 0]          # temps
7 U = data[:, 1:]         # solution numérique
8 X = np.linspace(0, 500, U.shape[1]) # espace
9
10 # Tracé heatmap X-T
11 plt.figure(figsize=(10,6))
12 plt.pcolormesh(X, T, U, shading='auto', cmap='hot') # Colormap "hot"
13 cbar = plt.colorbar()
14 cbar.set_label('Valeur de la solution U(X,T)', fontsize=12)
15
16 # Noms des axes
17 plt.xlabel('Position X (espace)', fontsize=12)
18 plt.ylabel('Temps T', fontsize=12)
19
20 # Titre
21 plt.title("Évolution du payoff du contrat et le prix de l'option d'achat",
22          fontsize=14)
23 plt.gca().invert_yaxis() # Optionnel: T augmente vers le haut si on
24                           commente cette ligne
25 plt.show()
```

Explication du code :

- On importe `numpy` pour gérer les tableaux et `matplotlib` pour tracer la heat map.
- On charge les données de `BS_implicit.txt` : la première colonne correspond au temps T et les colonnes suivantes à la solution numérique $U(X,T)$.
- Le vecteur spatial X est créé avec `linspace` pour correspondre à la dimension de la solution.
- La fonction `pcolormesh` crée la heat map, où chaque cellule est colorée selon la valeur de $U(X,T)$. La colormap `hot` met en évidence les zones de valeur élevée par des couleurs plus chaudes.
- On ajoute une barre de couleur (`colorbar`) pour indiquer les valeurs exactes de $U(X,T)$.
- Les axes et le titre sont définis avec `xlabel`, `ylabel` et `title`.
- L'option `invert_yaxis` inverse l'axe des temps pour une lecture plus intuitive (temps croissant vers le haut).

Graphe en Heatmap

Analyse du graphique de type heatmap La carte thermique ci-dessus illustre l'évolution temporelle et spatiale de la solution $U(X,T)$ de l'équation de Black-Scholes. Les zones claires (jaunes/blanches) correspondent à des valeurs élevées du prix de l'option, observées pour des valeurs de X importantes et des temps proches de l'échéance. Lorsque

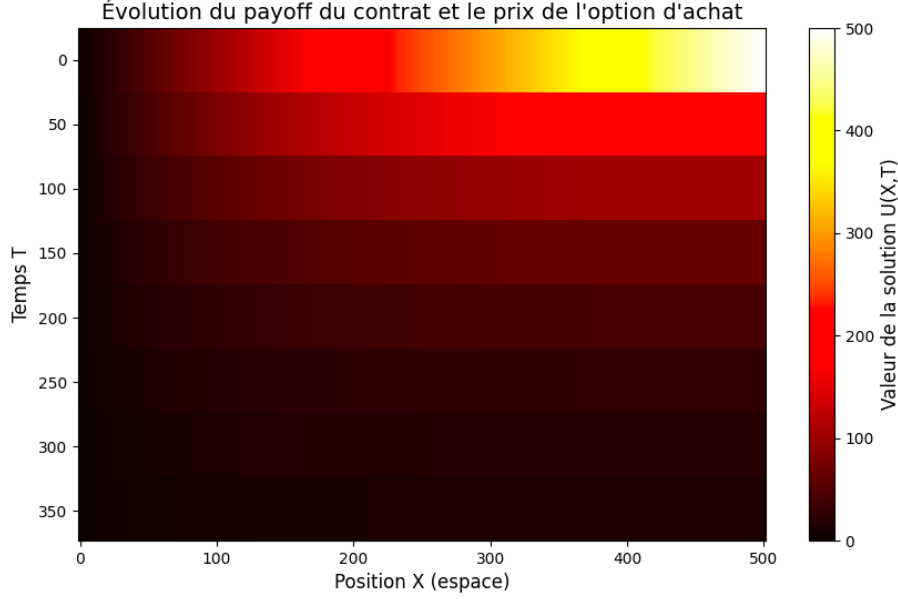


FIGURE 6 – Évolution du payoff du contrat et le prix de l'option d'achat.

le temps augmente, les couleurs deviennent plus sombres, traduisant la diminution du prix de l'option au fil du temps, conformément à l'effet de la *décote temporelle*. Pour les petites valeurs de X , la solution reste proche de zéro, indiquant que le call est hors de la monnaie (*out of the money*). L'évolution régulière des couleurs sans oscillations confirme la stabilité du schéma implicite utilisé pour la résolution numérique.

5 Méthodes itératives pour les matrices creuses : application à la température optimale d'un four pour la cuisson d'un gâteau

On étudie dans cette partie le champ de température d'un gâteau en cours de cuisson dans un four, à partir de la valeur connue des résistances électriques. On suppose le phénomène stationnaire, c'est-à-dire indépendant du temps (ce qui correspond au cas où le four a atteint la température imposée, le gâteau étant déjà à l'intérieur). Le but du projet est la résolution, par diverses méthodes itératives, du système linéaire $Ax = b$ issu de la discrétisation par différences finies des équations définissant notre problème.

Le four est représenté par le domaine $\Omega =]0, 1[^2$, de frontière $\partial\Omega$, et l'on note $\mathbf{n}$ le vecteur normal extérieur unitaire. On suppose que la frontière est décomposée en deux parties disjointes : $\partial\Omega = \partial\Omega_D \cup \partial\Omega_N$, avec $\partial\Omega_D$ de mesure non nulle. Le gâteau est représenté par le domaine $G = [0.2, 0.8] \times [0.3, 0.4]$.

Étant donnée une fonction $\mathbf{v} = (v_1, v_2) : \Omega \rightarrow \mathbb{R}^2$, on définit l'opérateur divergence par $\text{div } \mathbf{v} = \frac{\partial v_1}{\partial x_1} + \frac{\partial v_2}{\partial x_2}$. Le problème de diffusion de la chaleur s'écrit alors comme suit :

trouver la fonction température $T : \Omega \rightarrow \mathbb{R}$, de classe $\mathcal{C}^4(\Omega)$, telle que

$$\begin{cases} -\operatorname{div}(\rho \nabla T) = 0 & \text{dans } \Omega, \\ T = T_D & \text{sur } \partial\Omega_D, \\ \nabla T \cdot \mathbf{n} = 0 & \text{sur } \partial\Omega_N. \end{cases} \quad (11)$$

On note $\rho : \Omega \rightarrow \mathbb{R}_+^*$ le coefficient de conductivité thermique, et T_D la température imposée sur le bord $\partial\Omega_D$ (par exemple $T_D = 180^\circ$ sur le bord supérieur du four et $T_D = 20^\circ$ sur le bord inférieur). Les conditions aux limites de Neumann sont nulles, traduisant une isolation thermique parfaite du four.

Afin de discrétiser le problème par la méthode des différences finies, on introduit un maillage structuré de $\bar{\Omega}$, avec $N \in \mathbb{N}^*$ et un pas $h = \frac{1}{N+1}$ dans chaque direction. On définit les points du maillage $x_{i,j} = (ih, jh)$, $i, j = 0, \dots, N+1$, et l'on cherche une approximation $T_{i,j}$ de la température $T(x_{i,j})$.

5.1 Première partie : La discrétisation par différences finies

(1.1) On considère le coefficient de conductivité $\rho(x_1, x_2) = 1$ pour tout $(x_1, x_2) \in \Omega$. Montrer que, pour $T \in \mathcal{C}^4(\Omega)$ et pour h suffisamment petit, le système d'équations s'écrit :

$$\begin{cases} \frac{4T_{i,j} - T_{i-1,j} - T_{i+1,j} - T_{i,j-1} - T_{i,j+1}}{h^2} = 0, & \forall 1 \leq i, j \leq N, \\ T_{i,0} = 20, \quad T_{i,N+1} = 180, & \forall i = 0, \dots, N+1, (\text{C.L. de Dirichlet sur } \partial\Omega_D), \\ T_{-1,j} = T_{1,j}, \quad T_{N+2,j} = T_{N,j}, & \forall j = 1, \dots, N, (\text{C.L. de Neumann sur } \partial\Omega_N). \end{cases}$$

On a donc : Avec $\rho = 1$, l'équation $-\operatorname{div}(\rho \nabla T) = 0$ se réduit à $-\Delta T = 0$ dans Ω , c'est-à-dire

$$-\left(\frac{\partial^2 T}{\partial x_1^2} + \frac{\partial^2 T}{\partial x_2^2} \right) = 0.$$

On considère un maillage uniforme de pas $h = \frac{1}{N+1}$, et les points $x_{i,j} = (ih, jh)$, $i, j = 0, \dots, N+1$. On note $T_{i,j} \approx T(x_{i,j})$.

Comme $T \in \mathcal{C}^4(\Omega)$, on peut utiliser les développements de Taylor. Pour h suffisamment petit, on a :

$$\begin{aligned} \frac{\partial^2 T}{\partial x_1^2}(x_{i,j}) &\simeq \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{h^2}, \\ \frac{\partial^2 T}{\partial x_2^2}(x_{i,j}) &\simeq \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{h^2}. \end{aligned}$$

En les sommant, on obtient :

$$\Delta T(x_{i,j}) \simeq \frac{T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j}}{h^2}.$$

L'équation $-\Delta T = 0$ donne donc, pour tout point intérieur $1 \leq i, j \leq N$,

$$\frac{4T_{i,j} - T_{i-1,j} - T_{i+1,j} - T_{i,j-1} - T_{i,j+1}}{h^2} = 0.$$

Il s'agit du schéma à cinq points classique pour l'équation de Laplace.

Conditions de Dirichlet (bords haut et bas) On impose directement la température :

$$T_{i,0} = 20, \quad T_{i,N+1} = 180, \quad i = 0, \dots, N+1.$$

Conditions de Neumann (bords latéraux isolés) La condition $\nabla T \cdot \mathbf{n} = 0$ signifie que la dérivée normale est nulle.

À gauche ($x_1 = 0$) :

$$\frac{T_{1,j} - T_{-1,j}}{2h} = 0 \implies T_{-1,j} = T_{1,j}.$$

À droite ($x_1 = 1$) :

$$\frac{T_{N+2,j} - T_{N,j}}{2h} = 0 \implies T_{N+2,j} = T_{N,j},$$

pour $j = 1, \dots, N$.

Conclusion

Sous l'hypothèse $T \in \mathcal{C}^4(\Omega)$ et pour h suffisamment petit, la discrétisation par différences finies conduit au système :

$$\begin{cases} \frac{4T_{i,j} - T_{i-1,j} - T_{i+1,j} - T_{i,j-1} - T_{i,j+1}}{h^2} = 0, & \forall 1 \leq i, j \leq N, \\ T_{i,0} = 20, \quad T_{i,N+1} = 180, & \forall i = 0, \dots, N+1, (\text{C.L. de Dirichlet sur } \partial\Omega_D), \\ T_{-1,j} = T_{1,j}, \quad T_{N+2,j} = T_{N,j}, & \forall j = 1, \dots, N, (\text{C.L. de Neumann sur } \partial\Omega_N). \end{cases}$$

Ce système s'écrit sous la forme matricielle $Ax = b$, qui constitue la base de la suite du projet.

(1.2) Montrer que, pour $T \in \mathcal{C}^4(\Omega)$ et $\rho \in \mathcal{C}^2(\Omega)$, et pour h suffisamment petit, on a :

$$\begin{aligned} \frac{[\rho(x_1 + \frac{h}{2}, x_2) + \rho(x_1 - \frac{h}{2}, x_2)] T(x_1, x_2) - \rho(x_1 + \frac{h}{2}, x_2) T(x_1 + h, x_2) - \rho(x_1 - \frac{h}{2}, x_2) T(x_1 - h, x_2)}{h^2} \\ = -\frac{\partial}{\partial x_1} \left(\rho \frac{\partial T}{\partial x_1} \right) (x_1, x_2) + \mathcal{O}(h^2). \end{aligned}$$

Soient $T \in \mathcal{C}^4(\Omega)$ et $\rho \in \mathcal{C}^2(\Omega)$. On se place en un point $(x_1, x_2) \in \Omega$.

Pour la température, on a les développements de Taylor :

$$T(x_1 \pm h, x_2) = T \pm h \partial_{x_1} T + \frac{h^2}{2} \partial_{x_1 x_1} T \pm \frac{h^3}{6} \partial_{x_1 x_1 x_1} T + \mathcal{O}(h^4).$$

Pour la conductivité thermique :

$$\rho\left(x_1 \pm \frac{h}{2}, x_2\right) = \rho \pm \frac{h}{2} \partial_{x_1} \rho + \frac{h^2}{8} \partial_{x_1 x_1} \rho + \mathcal{O}(h^3).$$

On cherche à approximer l'opérateur :

$$-\frac{\partial}{\partial x_1} \left(\rho \frac{\partial T}{\partial x_1} \right) (x_1, x_2).$$

On utilise une différence centrée pour la dérivée selon x_1 :

$$-\partial_{x_1}(\rho \partial_{x_1} T) \simeq -\frac{(\rho \partial_{x_1} T)(x_1 + \frac{h}{2}, x_2) - (\rho \partial_{x_1} T)(x_1 - \frac{h}{2}, x_2)}{h}.$$

Pour h petit, on a :

$$\begin{aligned}\partial_{x_1} T(x_1 + \frac{h}{2}, x_2) &\simeq \frac{T(x_1 + h, x_2) - T(x_1, x_2)}{h}, \\ \partial_{x_1} T(x_1 - \frac{h}{2}, x_2) &\simeq \frac{T(x_1, x_2) - T(x_1 - h, x_2)}{h}.\end{aligned}$$

Il en résulte :

$$\begin{aligned}(\rho \partial_{x_1} T)(x_1 + \frac{h}{2}, x_2) &\simeq \rho(x_1 + \frac{h}{2}, x_2) \frac{T(x_1 + h, x_2) - T(x_1, x_2)}{h}, \\ (\rho \partial_{x_1} T)(x_1 - \frac{h}{2}, x_2) &\simeq \rho(x_1 - \frac{h}{2}, x_2) \frac{T(x_1, x_2) - T(x_1 - h, x_2)}{h}.\end{aligned}$$

En injectant dans la différence centrée, on obtient :

$$\begin{aligned}\frac{[\rho(x_1 + \frac{h}{2}, x_2) + \rho(x_1 - \frac{h}{2}, x_2)] T(x_1, x_2) - \rho(x_1 + \frac{h}{2}, x_2) T(x_1 + h, x_2) - \rho(x_1 - \frac{h}{2}, x_2) T(x_1 - h, x_2)}{h^2} \\ = -\frac{\partial}{\partial x_1} \left(\rho \frac{\partial T}{\partial x_1} \right) (x_1, x_2) + \mathcal{O}(h^2).\end{aligned}$$

Ainsi, l'approximation est d'ordre 2, puisque $T \in C^4(\Omega)$ et $\rho \in C^2(\Omega)$.

En inversant les rôles de x_1 et x_2 , on obtient une approximation analogue pour

$$-\partial_{x_2}(\rho \partial_{x_2} T).$$

Deplus, on note les points intermédiaires du maillage :

$$x_{i \pm \frac{1}{2}, j} = (ih \pm \frac{h}{2}, jh), \quad x_{i, j \pm \frac{1}{2}} = (ih, jh \pm \frac{h}{2}).$$

On note :

$$T_{i,j} \approx T(x_{i,j}), \quad \rho_{i \pm \frac{1}{2}, j} = \rho(x_{i \pm \frac{1}{2}, j}), \quad \rho_{i, j \pm \frac{1}{2}} = \rho(x_{i, j \pm \frac{1}{2}}).$$

Vérification du système discret

On considère l'équation continue

$$-\operatorname{div}(\rho \nabla T) = 0 \quad \text{dans } \Omega.$$

Par définition, on a :

$$-\operatorname{div}(\rho \nabla T) = -\frac{\partial}{\partial x_1} \left(\rho \frac{\partial T}{\partial x_1} \right) - \frac{\partial}{\partial x_2} \left(\rho \frac{\partial T}{\partial x_2} \right).$$

On discrétise séparément chacun de ces deux termes au point du maillage

$$x_{i,j} = (ih, jh), \quad 1 \leq i, j \leq N.$$

(1) Discrétisation du terme selon x_1

D'après la question (1.2), pour $T \in C^4(\Omega)$ et $\rho \in C^2(\Omega)$, on obtient :

$$-\frac{\partial}{\partial x_1} \left(\rho \frac{\partial T}{\partial x_1} \right) (x_{i,j}) = \frac{(\rho_{i+\frac{1}{2},j} + \rho_{i-\frac{1}{2},j})T_{i,j} - \rho_{i+\frac{1}{2},j}T_{i+1,j} - \rho_{i-\frac{1}{2},j}T_{i-1,j}}{h^2} + \mathcal{O}(h^2).$$

Ce terme couple donc :

- le point central $T_{i,j}$,
- les voisins gauche $T_{i-1,j}$ et droit $T_{i+1,j}$.

(2) Discrétisation du terme selon x_2

De manière analogue, en inversant les rôles de x_1 et x_2 , on obtient :

$$-\frac{\partial}{\partial x_2} \left(\rho \frac{\partial T}{\partial x_2} \right) (x_{i,j}) = \frac{(\rho_{i,j+\frac{1}{2}} + \rho_{i,j-\frac{1}{2}})T_{i,j} - \rho_{i,j+\frac{1}{2}}T_{i,j+1} - \rho_{i,j-\frac{1}{2}}T_{i,j-1}}{h^2} + \mathcal{O}(h^2).$$

Ce terme couple :

- le point central $T_{i,j}$,
- les voisins bas $T_{i,j-1}$ et haut $T_{i,j+1}$.

En additionnant les deux approximations précédentes et en négligeant les termes en $\mathcal{O}(h^2)$, on obtient :

$$\begin{aligned} -\operatorname{div}(\rho \nabla T)(x_{i,j}) &= \frac{(\rho_{i+\frac{1}{2},j} + \rho_{i-\frac{1}{2},j} + \rho_{i,j+\frac{1}{2}} + \rho_{i,j-\frac{1}{2}})T_{i,j}}{h^2} \\ &\quad - \frac{\rho_{i+\frac{1}{2},j}T_{i+1,j} + \rho_{i-\frac{1}{2},j}T_{i-1,j} + \rho_{i,j+\frac{1}{2}}T_{i,j+1} + \rho_{i,j-\frac{1}{2}}T_{i,j-1}}{h^2}. \end{aligned}$$

On pose :

$$\tilde{\rho}_{i,j} = \rho_{i+\frac{1}{2},j} + \rho_{i-\frac{1}{2},j} + \rho_{i,j+\frac{1}{2}} + \rho_{i,j-\frac{1}{2}}.$$

L'équation discrète devient :

$$\frac{\tilde{\rho}_{i,j}T_{i,j} - \rho_{i+\frac{1}{2},j}T_{i+1,j} - \rho_{i-\frac{1}{2},j}T_{i-1,j} - \rho_{i,j+\frac{1}{2}}T_{i,j+1} - \rho_{i,j-\frac{1}{2}}T_{i,j-1}}{h^2} = 0, \quad 1 \leq i, j \leq N.$$

Dirichlet

$$T_{i,0} = 20, \quad T_{i,N+1} = 180, \quad i = 0, \dots, N+1.$$

Neumann homogènes

$$\partial_{x_1} T = 0 \implies T_{-1,j} = T_{1,j}, \quad T_{N+2,j} = T_{N,j}, \quad j = 1, \dots, N.$$

Pour assurer la cohérence du schéma, on prolonge ρ de façon symétrique :

$$\rho_{-\frac{1}{2},j} = \rho_{\frac{1}{2},j}, \quad \rho_{N+\frac{3}{2},j} = \rho_{N+\frac{1}{2},j}.$$

Pour l'application du gâteau G dans le four on prendra :

$$\rho(x_1, x_2) = \begin{cases} 100, & (x_1, x_2) \in G = [0.2, 0.8] \times [0.3, 0.4], \\ 1, & (x_1, x_2) \in \Omega \setminus G. \end{cases}$$

(1.3) Nous allons numéroter les inconnues (et les équations) suivant l'ordre lexicographique :

$$T_{0,0}, T_{1,0}, \dots, T_{N+1,0}, T_{0,1}, T_{1,1}, \dots, T_{N+1,1}, \dots$$

On détaille point par point pour $N = 3$. En numérotant les inconnues selon l'ordre lexicographique, on a :

$$T_{i,j} \mapsto k = i + (N+2)j,$$

avec

$$i = 0, \dots, 4, \quad j = 0, \dots, 4 \quad (\text{car } N + 2 = 5).$$

L'ordre est donc :

$$\begin{aligned} j = 0 : & \quad T_{0,0}, T_{1,0}, T_{2,0}, T_{3,0}, T_{4,0}, \\ j = 1 : & \quad T_{0,1}, T_{1,1}, T_{2,1}, T_{3,1}, T_{4,1}, \\ & \quad \vdots \\ j = 4 : & \quad T_{0,4}, T_{1,4}, T_{2,4}, T_{3,4}, T_{4,4}. \end{aligned}$$

Il y a donc 25 inconnues au total.

Pour les points intérieurs ($1 \leq i, j \leq 3$), on obtient l'équation discrète :

$$\frac{\tilde{\rho}_{i,j}T_{i,j} - \rho_{i+\frac{1}{2},j}T_{i+1,j} - \rho_{i-\frac{1}{2},j}T_{i-1,j} - \rho_{i,j+\frac{1}{2}}T_{i,j+1} - \rho_{i,j-\frac{1}{2}}T_{i,j-1}}{h^2} = 0.$$

Ces équations correspondent aux lignes centrales de la matrice.

Pour les conditions de Dirichlet (bas et haut)

Bas ($j = 0$) :

$$T_{i,0} = 20.$$

Haut ($j = 4$) :

$$T_{i,4} = 180.$$

Dans la matrice associée :

- coefficient diagonal = 1,
- autres coefficients = 0,
- second membre = 20 ou 180.

Pour les onditions de Neumann (gauche et droite)

Gauche :

$$T_{-1,j} = T_{1,j}.$$

Droite :

$$T_{5,j} = T_{3,j}.$$

Ces conditions sont intégrées via les points fictifs.

La matrice $A \in \mathbb{R}^{25 \times 25}$ a la structure par blocs suivante :

$$A = \begin{pmatrix} I_5 & 0 & 0 & 0 & 0 \\ * & B & * & 0 & 0 \\ 0 & * & B & * & 0 \\ 0 & 0 & * & B & * \\ 0 & 0 & 0 & 0 & I_5 \end{pmatrix}.$$

Ici :

- I_5 correspond aux conditions de Dirichlet,
- B est le bloc des équations intérieures,
- $*$ représente les couplages verticaux.

Pour un point intérieur (i, j) , la ligne correspondante contient :

$$A_{k,k} = \frac{\tilde{\rho}_{i,j}}{h^2}, \quad A_{k,k\pm 1} = -\frac{\rho_{i\pm\frac{1}{2},j}}{h^2}, \quad A_{k,k\pm(N+2)} = -\frac{\rho_{i,j\pm\frac{1}{2}}}{h^2}.$$

Il s'agit d'un schéma pondéré à cinq points.

Et pour le second membre b :

Pour $j = 0$:

$$b_k = 20.$$

Pour $j = 4$:

$$b_k = 180.$$

Pour les autres indices :

$$b_k = 0.$$

$\Rightarrow$ seuls les premiers et les derniers $N + 2 = 5$ éléments de b sont non nuls.

Remarque : Symétrie de la matrice A

Les blocs intérieurs sont symétriques, car les coefficients $\rho_{i+\frac{1}{2},j}$ apparaissent symétriquement entre (i, j) et $(i + 1, j)$. Cependant, les lignes de Dirichlet cassent la symétrie globale :

A n'est pas symétrique.

Pour obtenir une matrice symétrique, il faut éliminer les inconnues de Dirichlet. Le système réduit contient $(N + 1)^2$ inconnues, numérotées par

$$T_{i,j} = T[i + j(N + 1)].$$

La matrice obtenue est alors symétrique et définie positive.

Pour $N = 3$, le nombre d'inconnues est

$$(N + 1)^2 = 16.$$

On numérote les inconnues $T_{i,j}$ avec $0 \leq i, j \leq 3$ de la manière suivante :

$j \backslash i$	0	1	2	3
0	T_0	T_1	T_2	T_3
1	T_4	T_5	T_6	T_7
2	T_8	T_9	T_{10}	T_{11}
3	T_{12}	T_{13}	T_{14}	T_{15}

Pour tout $0 \leq i, j \leq 3$, on écrit :

$$\frac{1}{h^2} \left(\tilde{\rho}_{i,j} T_{i,j} - \rho_{i+\frac{1}{2},j} T_{i+1,j} - \rho_{i-\frac{1}{2},j} T_{i-1,j} - \rho_{i,j+\frac{1}{2}} T_{i,j+1} - \rho_{i,j-\frac{1}{2}} T_{i,j-1} \right) = b_{i,j}.$$

Conventions aux limites

— Si un indice sort du domaine, on impose une condition de Neumann :

$$T_{-1,j} = T_{1,j}, \quad T_{N+1,j} = T_{N-1,j}.$$

— Si un point correspond à une condition de Dirichlet, le terme est reporté dans le second membre.

Second membre b

Seuls les points adjacents aux frontières de Dirichlet produisent un terme non nul dans b .

— Pour $j = 0$ (frontière inférieure avec $T = 20$) :

$$b_{i,0} = \frac{\rho_{i,-\frac{1}{2}}}{h^2} 20.$$

— Pour $j = 3$ (frontière supérieure avec $T = 180$) :

$$b_{i,3} = \frac{\rho_{i,\frac{7}{2}}}{h^2} 180.$$

— Pour les autres points :

$$b_{i,j} = 0.$$

$\Rightarrow$ Seuls $2(N + 1) = 8$ coefficients du vecteur b (ceux correspondant aux premières et dernières lignes) sont non nuls.

Matrice A La matrice A possède une structure en blocs :

$$A = \begin{pmatrix} B & C & 0 & 0 \\ C^T & B & C & 0 \\ 0 & C^T & B & C \\ 0 & 0 & C^T & B \end{pmatrix},$$

où :

- B est une matrice tridiagonale correspondant aux interactions selon x ,
- C représente les couplages selon la direction y .

$\Rightarrow$ La matrice A est symétrique. En effet, le coefficient reliant deux nœuds voisins est de la forme $-\frac{\rho_{i+\frac{1}{2},j}}{h^2}$, et apparaît de manière identique et symétrique dans les deux équations associées aux nœuds concernés.

Définie positivité

Pour tout vecteur $x \neq 0$, on a :

$$x^T A x = \frac{1}{h^2} \sum_{i,j} \left[\rho_{i+\frac{1}{2},j} (x_{i+1,j} - x_{i,j})^2 + \rho_{i,j+\frac{1}{2}} (x_{i,j+1} - x_{i,j})^2 \right] > 0,$$

car $\rho > 0$ et il s'agit d'une somme de carrés. Ainsi, la matrice A est définie positive.

Remarque : Dans les sections suivantes, le code étant très long, seule la ligne d'en-tête des fonctions sera présentée. Pour obtenir l'ensemble des codes du projet, veuillez consulter le lien à la section 6.

(1.4) Constituer la classe `matricebande` qui hérite les champs et les méthodes de la classe `matrice` et qui mettra en place le stockage bande de A . On rappelle que pour A symétrique, on peut stocker juste les 3 diagonales non nulles de sa partie triangulaire inférieure à l'aide d'une matrice rectangulaire `ATAB` et d'un vecteur d'entiers `IND` qui donne la position des diagonales de A . La classe `matricebande` ajoutera le champ `IND` aux champs de la classe mère `matrice` et plusieurs fonctions membres décrites dans les questions suivantes.

Matrice A complète (16×16)

On factorise $\frac{1}{h^2} = 16$. La matrice ci-dessous est déjà divisée par h^2 , donc les coefficients sont 4 et -1 .

$$A = \begin{pmatrix} \boxed{4} & -1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & \boxed{4} & -1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & \boxed{4} & -1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & \boxed{4} & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & \boxed{4} & -1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & -1 & \boxed{4} & -1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & -1 & \boxed{4} & -1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & -1 & \boxed{4} & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & \boxed{4} & -1 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & -1 & \boxed{4} & -1 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & -1 & \boxed{4} & -1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & -1 & \boxed{4} & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & \boxed{4} & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & -1 & \boxed{4} & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & -1 & \boxed{4} & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & -1 & \boxed{4} \end{pmatrix}$$

FIGURE 7 – Matrice A

Voici le code :

```
1 class matricebande : public matrice{
2 private:
3     vector<int> indice;
4 public:
5     matricebande() {};
```

// constructeur par

default

```
6     matricebande(int,int);
```

// constructeur en

donnant 2 dimensions

```
7     matricebande(const matricebande&);
```

// constructeur par

recopie

```
8     ~matricebande() {};
```

// destructeur par défaut

```
9
10    vector<int> ind() const {return this->indice;}    // retourne le
    champ priv indice
11    vector<int> ind(vector<int>);
```

// remplit le

champ priv indice

```
12
13    matricebande laplacien(int);
```

// assemblage du

laplacien TP3, question (1.1)

```
14    matricebande laplacien_rho(int);
15    vector<double> prod(const vector<double>& x) const;
```

```
16
17    friend vector<double> smbr(int N);
18    friend vector<double> smbr_rho(int N);
19
20 };
21
22 // D clarations des seconds membres (1.9)
23 vector<double> smbr(int N);
24 vector<double> smbr_rho(int N);
```

(1.5) La fonction membre de la classe `matricebande` nommée `laplacien` qui prend en entrée un entier $N > 1$ et remplit les tableaux `ATAB` et `IND` correspondants à la partie triangulaire inférieure de A correspondant à la question (1.1).

Voici le résultat quand on lance le code :

```

1 N = 3
2 matrice du laplacien avant sym trisation
3 0 0 64
4 0 -16 64
5 0 -16 64
6 0 -16 64
7 0 -32 64
8 -16 0 64
9 -16 -16 64
10 -16 -16 64
11 -16 -16 64
12 -16 -32 64
13 -16 0 64
14 -16 -16 64
15 -16 -16 64
16 -16 -16 64
17 -16 -32 64
18 Vecteur IND :
19 -5 -1 0
20 Matrice bande ATAB :
21 0 0 32
22 0 -16 64
23 0 -16 64
24 0 -16 64
25 0 -16 32
26 -8 0 32
27 -16 -16 64
28 -16 -16 64
29 -16 -16 64
30 -8 -16 32
31 -8 0 32
32 -16 -16 64
33 -16 -16 64
34 -16 -16 64
35 -8 -16 32

```

(1.6) La fonction membre de la classe `matricebande` nommée `laplacien_rho` qui prend en entrée un entier $N > 1$ et remplit les tableaux `ATAB` et `IND` correspondants à la partie triangulaire inférieure de la matrice A obtenue à la question (1.2).

Voici le résultat quand on lance le code :

```

1 N = 3
2 matrice du laplacien_rho avant sym trisation
3 0 0 64
4 0 -16 3232
5 0 -16 3232
6 0 -16 3232
7 0 -16 64
8 -16 0 64
9 -1600 -16 1648
10 -1600 -16 1648
11 -1600 -16 1648

```

```

12 -16 -16 64
13 -16 0 64
14 -16 -16 64
15 -16 -16 64
16 -16 -16 64
17 -16 -16 64
18 Vecteur IND :
19 -5 -1 0
20 Matrice bande ATAB :
21 0 0 32
22 0 -16 3232
23 0 -16 3232
24 0 -16 3232
25 0 -8 32
26 -8 0 32
27 -1600 -16 1648
28 -1600 -16 1648
29 -1600 -16 1648
30 -8 -8 32
31 -8 0 32
32 -16 -16 64
33 -16 -16 64
34 -16 -16 64
35 -8 -8 32

```

(1.7) La fonction membre de la classe `matricebande` qui surcharge l'opérateur `*` et qui prend en entrée un vecteur x (vérifier sa taille) afin de retourner le vecteur $y = Ax$ avec A symétrique donnée en stockage bande.

Comme on a déjà l'opérateur `*` dans la class `matrice` que la class `matricebande` a hérité donc on redefinit dans un autre nom : `prod`. Voci le head de la fonction `prod`.

```

1 vector<double> matricebande::prod(const vector<double>& x) const
2 {

```

(1.8) On testera dans le programme principal l'assemblage de la matrice A et le produit Ax en affichant le résultat à l'écran (prendre par exemple $x = (1, \dots, 1)^T$ avec N petit).

Voici le résultat quand on lance le code :

```

Vecteur x = (1,...,1) :
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

// pour laplacien
Vecteur y = A x :
8 16 16 16 8 0 0 0 0 0 8 16 16 16 8

// pu laplacien_rho
Vecteur y = A2 x2 :
8 1600 1600 1608 16 0 0 0 8 8 8 16 16 24 16

```

(1.9) Les fonctions amies de la classe `matricebande` nommées respectivement `smbr` et `smbr_rho` qui remplissent le second membre b correspondants respectivement aux questions (1.1) et (1.2).

Voici le head de la fonction `smb` et `smb_rho`.

```
1 vector<double> smb(int n){}
2 vector<double> smb_rho(int n){}
```

5.2 Deuxième partie : La méthode de gradient à pas optimal et à pas fixe.

Considérer le système linéaire donné dans le cas (1.1). Partant de $x_0 = 0$, $r_0 = b - Ax_0$, la méthode de gradient à pas optimal (steepest descent) est donnée par les récurrences

$$x_{k+1} = x_k + \alpha_k r_k, \quad r_{k+1} = r_k - \alpha_k A r_k, \quad \alpha_k = \frac{r_k^T r_k}{r_k^T A r_k}.$$

On propose d'arrêter les itérations quand $\|r_k\| \leq \text{tol} \|r_0\|$, avec $\text{tol} = 10^{-4}$, sachant que l'on ne souhaite pas effectuer plus de $10 \cdot \text{size}(x_0, 1)$ itérations. Aussi, on ne souhaite pas que la taille de $\|r_k\|$ augmente trop : on arrête également les itérations (avec un message d'erreur `OVERFLOW`) quand $\|r_k\| \geq \|r_0\|/\text{tol}$.

(2.1) La fonction membre de la classe `matricebande` nommée `steepest_descent` pour la méthode de gradient à pas optimal, en prenant comme entrée le vecteur b et le paramètre `tol`, comme entrée-sortie le vecteur x (qui contient en entrée le vecteur x_0 et en sortie le dernier itéré x_k) ainsi qu'en sortie le nombre d'itérations effectuées pour atteindre la condition d'arrêt et un vecteur qui stocke les valeurs $\|r_k\|/\|b\|$ calculées à chaque itération k .

Voici le résultat quand on lance le code :

```
k = 0 ||r_k|| / ||b|| = 1
k = 1 ||r_k|| / ||b|| = 0.653613
k = 2 ||r_k|| / ||b|| = 0.455128
k = 3 ||r_k|| / ||b|| = 0.412339
k = 4 ||r_k|| / ||b|| = 0.328655
-----
k = 66 ||r_k|| / ||b|| = 9.61904e-05
Nombre d'iterations : 66
Dernier residu relatif : 9.61904e-05
```

(2.2) On sait que l'algorithme de Richardson donne $\alpha_{\text{opt}} = \frac{2}{\lambda_1 + \lambda_n}$ (voir feuille de TD), où λ_1 (resp. λ_n) désigne la plus petite (resp. la plus grande) valeur propre de la matrice A . Dans notre cas, on approche $\frac{1}{\alpha_{\text{opt}}}$ par le plus grand coefficient de la diagonale de A . On écrira une fonction membre de la classe `matricebande`, nommée `gradient_pas_fixe`, implémentant cette méthode.

Voici le résultat quand on lance le code :

```
alpha_opt = 0.015625

j    alpha        nb_iter
```

1	0.003125	150
2	0.00625	150
3	0.009375	122
4	0.0125	91
5	0.015625	72
6	0.01875	150

On a obtenu le tableau du nombre d'itérations pour atteindre la condition d'arrêt

Méthode	j	$\alpha = \frac{j \alpha_{\text{opt}}}{5}$	Nombre d'itérations
Gradient à pas fixe	1	0.003125	150 (max)
Gradient à pas fixe	2	0.00625	150 (max)
Gradient à pas fixe	3	0.009375	122
Gradient à pas fixe	4	0.0125	91
Gradient à pas fixe	5	0.015625	72
Gradient à pas fixe	6	0.01875	150 (divergence)
Steepest descent	–	pas optimal automatique	66

TABLE 2 – Influence du choix du pas α sur la convergence des méthodes de gradient

Voici le graph comaraison de 2 méthodes :

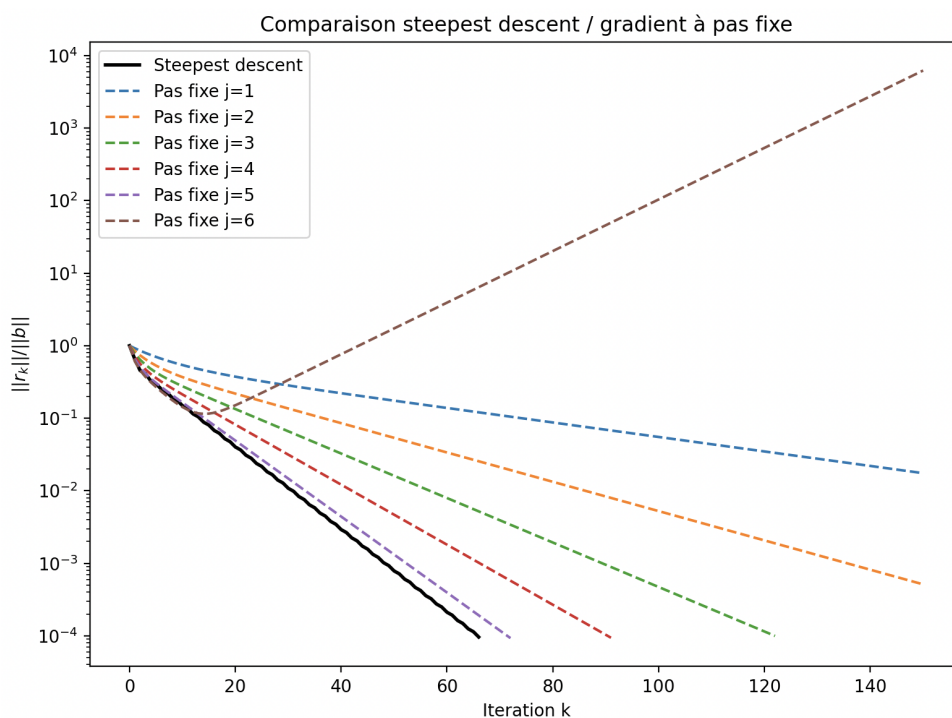


FIGURE 8 – la fonction $k \rightarrow \frac{\|r_k\|}{\|b\|}$ sur une échelle semi-logarithmique pour les deux méthodes, avec les différents choix du α fixe.

Observation :

La représentation semi-logarithmique du résidu relatif met en évidence la forte dépendance de la méthode du gradient à pas fixe au choix du paramètre α . Pour des valeurs

proches de α_{opt} , la convergence est la plus rapide. Par exemple des valeurs faibles du pas ($\alpha_1 = 0.003125$ et $\alpha_2 = 0.00625$), la méthode converge très lentement et atteint le nombre maximal d'itérations autorisé (150) sans satisfaire la condition d'arrêt. Lorsque le pas augmente ($\alpha_3, \alpha_4, \alpha_5$), le nombre d'itérations diminue significativement, ce qui met en évidence une accélération de la convergence. Le meilleur compromis est obtenu pour $\alpha_5 = 0.015625$, avec seulement 72 itérations.

En revanche, lorsque le pas devient trop grand ($\alpha_6 = 0.01875$), la méthode devient instable et diverge, ce qui illustre la sensibilité du gradient à pas fixe au choix du α .

On a : $\lambda_{\text{max}} \approx \max_i A_{ii}$, d'où $\alpha_{\text{opt}} \approx \frac{1}{\lambda_{\text{max}}}$. Pour $j = 6$, on obtient :

$$\alpha = \frac{6}{5} \alpha_{\text{opt}} = \alpha = 1.2 \alpha_{\text{opt}} \approx \frac{1.2}{\lambda_{\text{max}}}.$$

Ce pas est trop grand et dépasse le seuil de stabilité, ce qui conduit à une divergence de la méthode. Un pas trop grand ($\alpha > \alpha_{\text{opt}}$) dépasse le seuil de stabilité et entraîne la divergence de la méthode.

La méthode de *steepest descent* avec pas optimal automatique présente de bien meilleures performances, avec seulement 66 itérations, tout en garantissant la convergence sans réglage manuel du pas. Ceci met en évidence l'intérêt d'un calcul adaptatif du pas, qui permet d'optimiser la vitesse de convergence tout en évitant les phénomènes de divergence.

⇒ **Résumé :**

- **Steepest descent :**
 - toujours stable.
 - pas optimal à chaque itération.
- **Gradient à pas fixe :**
 - très sensible à α .
 - peut diverger même après une phase de décroissance.

5.3 Troisième partie : la méthode PCG de gradient conjugué avec préconditionnement

Dans la méthode PCG, on résout itérativement le système $Ax = b$ préconditionné par $C = TT^t$ avec une matrice T triangulaire inférieure. Partant de $x = x_0$, $r = r_0 = b - Ax$, $z = C^{-1}r$, $p = p_0 = z$, et $\gamma = \gamma_0 = z^T r$, on calcule récursivement tant que $\gamma \geq \text{tol}^2 \gamma_0$:

$$q = Ap, \quad \alpha = \frac{\gamma}{p^T q}, \quad x = x + \alpha p, \quad r = r - \alpha q, \quad z = C^{-1}r, \quad \beta = \frac{1}{\gamma} \gamma, \quad \gamma = z^T r, \quad \beta = \beta \gamma, \quad p = z + \beta p.$$

On suppose que la matrice T triangulaire inférieure est stockée aussi au format bande à l'aide des tableaux **TTAB**, **TIND**. Nous allons discuter trois choix pour la matrice C :

1. La matrice identité $C = I$, et alors $T = I$;
2. La matrice diagonale $C = \text{diag}(A)$ obtenue par la diagonale de A , et alors $T = C^{1/2}$;
3. La matrice $\text{IC}(0)$ qui, pour le schéma à 5 points du Laplacien, se simplifie : on cherche $T = (T_{j,k})$ triangulaire inférieure avec des éléments non nuls aux mêmes positions que dans la partie triangulaire inférieure de A , de sorte que $A + \omega \text{diag}(A) - TT^t$ admet des zéros dans les positions correspondantes aux éléments non nuls de A .

On peut montrer, en tenant compte de la structure particulière de A , que pour $i > j$, $A_{i,j} \neq 0$:

$$(TT^t)_{i,j} = T_{i,j}T_{j,j}.$$

Par conséquent, les éléments non nuls de T sur chaque ligne i sont définis par

$$T_{i,j} = \frac{A_{i,j}}{T_{j,j}} \quad \text{pour } j < i, \quad T_{i,i} = \sqrt{A_{i,i}(1 + \omega) - \sum_{\ell=1}^{i-1} T_{i,\ell}^2},$$

sachant que $T_{i,i} = \text{TTAB}(\text{TIND}(2))$.

(3.1) La fonction membre de la classe `matricebande`, nommée `assemblageT(choix,  $\omega$ )`, qui, pour `choix` $\in \{1, 2, 3\}$ et un paramètre optionnel ω , remplit les tableaux `TTAB`, `TIND` pour l'un des trois préconditionneurs décrits ci-dessus.

Voci le head de la fonction `assemblageT(choix,  $\omega$ )`.

```
1 void matricebande::assemblageT(int choix, double omega){
```

On peut lancer un test pour voir ce qu'elle nous donne :

```
1 A.assemblageT(1);
2     cout << "TIND : ";
3     for (int k : A.get_TIND())
4         cout << k << " ";
5     cout << endl;
6     cout << "TTAB :\n";
7     cout << A.get_TTAB() << endl;
```

```
---- Choix 1 : C = I ----
```

```
TIND : 0
```

```
TTAB : 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
---- Choix 2 : C = diag(A) ----
```

```
TIND : 0
```

```
TTAB : 5.65685 8 8 8 5.65685 5.65685 8 8 8 5.65685 5.65685 8 8 8 5.65685
```

```
---- Choix 3 : IC(0) ----
```

```
TIND : -5 -1 0
```

```
TTAB :
```

```
0          0          5.65685
```

```
0        -2.82843    7.48331
```

```
0        -2.13809    7.70899
```

```
0        -2.0755     7.72608
```

```
0        -2.07091    5.26416
```

```
-1.41421 0          5.47723
```

```
-2.13809 -2.92119    7.13409
```

```
-2.0755   -2.24275    7.3934
```

```
-2.07091 -2.16409    7.41809
```

```
-1.51971 -2.15689    5.00383
```

```
-1.46059 0          5.46504
```

```
-2.24275 -2.9277     7.0992
```

```
-2.16409 -2.25378    7.36459
```

```
-2.15689 -2.17256    7.39106
```

```
-1.59878 -2.16478    4.97571
```

(3.2) La fonction membre de la classe `matricebande`, nommée `preconditionne(r)`, qui réalise une descente $Ty = r$ et une remontée $T^tz = y$.

Voci le head de la fonction `preconditionne(r)`

```
1 vector<double> matricebande::preconditionne(const vector<double>& r)
   const{
```

On peut lancer un test pour voir ce qu'elle nous donne :

```
1 A.assemblageT(1);
2     vector<double> z1 = A.preconditionne(r);
3     cout << "z = C^{-1} r (doit etre egal a r) :\n";
4     for (double zi : z1) cout << zi << " ";
5     cout << endl;
```

=== TEST preconditionne(r) ===

N = 3

Vecteur r :

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

---- Choix 1 : C = I ----

z = C^{-1} r (doit etre egal a r) :

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

---- Choix 2 : C = diag(A) ----

z = C^{-1} r :

0.03125 0.03125 0.046875 0.0625 0.15625 0.1875 0.109375 0.125 0.140625

0.3125 0.34375 0.1875 0.203125 0.21875 0.46875

---- Choix 3 : IC(0) ----

z = C^{-1} r :

0.380704 0.324384 0.352439 0.427866 0.514265 0.749049 0.626654 0.64905

0.769504 0.988846 0.774374 0.653832 0.668342 0.783226 1.10757

=== FIN TEST preconditionne ===

⇒ Remarque : Pour $N = 3$, le système comporte 15 inconnues et le vecteur test $r = (1, 2, \dots, 15)^T$ permet de valider simplement le préconditionneur. Pour le choix $C = I$, on obtient $z = r$, ce qui confirme la correction des phases de descente et de remontée. Avec $C = \text{diag}(A)$, les résultats correspondent à $z_i = r_i/A_{ii}$, validant le préconditionneur de Jacobi. Enfin, le préconditionneur $\text{IC}(0)$ fournit un vecteur z plus régulier, traduisant une meilleure approximation de A^{-1} , sans instabilité numérique.

(3.3) La fonction membre de la classe `matricebande`, nommée `PCG`, prenant comme entrée le paramètre `tol`, comme entrée-sortie le vecteur `x` (en entrée le vecteur x_0 et en sortie le dernier itéré x_k), les tableaux nécessaires pour stocker A et T , ainsi qu'en sortie le nombre d'itérations effectuées pour atteindre la condition d'arrêt et un vecteur stockant les valeurs $\frac{\|r_k\|}{\|b\|}$ calculées à chaque itération k .

Voci le head de la fonction `PCG`.

```
1 void matricebande::PCG(const vector<double>& b, vector<double>& x, double
   tol, int& nb_iter, vector<double>& res_hist) const
```

(3.4) Résoudre les systèmes linéaires donnés dans les parties (1.1) et (1.2), en prenant par exemple $N = 20$.

Voici le résultat quand on lance le code :

```
===== CAS (1.1) : rho = 1 =====
C = I           : 62
C = diag(A)     : 59
C = IC(0)       : 23

===== CAS (1.2) : rho variable (gateau) =====
C = I           : 248
C = diag(A)     : 83
C = IC(0)       : 26
```

Voici le tableau du nombre d'itérations du PCG pour les trois préconditionneurs considérés :

Cas	Préconditionneur	Itérations
$\rho = 1$	$C = I$	62
$\rho = 1$	$C = \text{diag}(A)$	59
$\rho = 1$	$C = \text{IC}(0)$	23
ρ variable	$C = I$	248
ρ variable	$C = \text{diag}(A)$	83
ρ variable	$C = \text{IC}(0)$	26

TABLE 3 – Nombre d'itérations du PCG pour différents préconditionneurs

⇒ Remarque : Pour le cas de conductivité constante, la méthode du gradient conjugué converge relativement rapidement et le préconditionneur $\text{IC}(0)$ permet de réduire le nombre d'itérations d'un facteur proche de 3. En revanche, dans le cas de conductivité variable présentant de forts contrastes, le conditionnement du système est fortement dégradé. Le préconditionneur $\text{IC}(0)$ permet alors une réduction très significative du nombre d'itérations, d'un facteur proche de 10 par rapport au gradient conjugué non préconditionné. Ces résultats confirment l'intérêt du préconditionnement, en particulier pour des problèmes hétérogènes.

Graph : Voici le graph de comparaison pour les PCG avec les différents préconditionneurs et différents choix du paramètre .

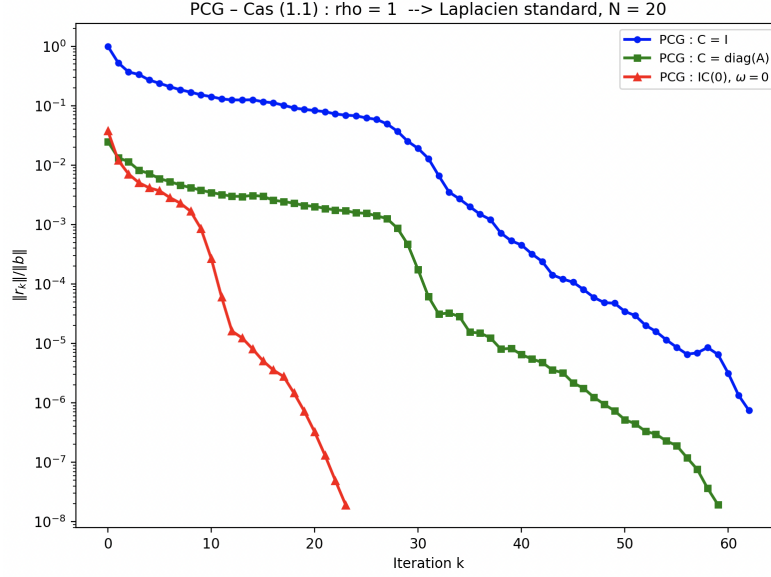


FIGURE 9 – La fonction $k \rightarrow \frac{\|r_k\|}{\|b\|}$ sur une échelle semi-logarithmique pour (1.1)

Remarque : Pour le (1.1)

PCG – $C = I$ Convergence la plus lente, nécessitant environ 60 itérations. Pente faible $\Rightarrow$ conditionnement moyen. Totalement cohérent avec le Laplacien 2D sans préconditionnement.

PCG – $C = \text{diag}(A)$ Plus rapide que $C = I$. Résidu décroît de manière plus régulière. Environ 55–60 itérations. Jacobi améliore légèrement le conditionnement, mais l’effet reste limité.

PCG – **IC(0)**, $\omega = 0$ Convergence très rapide, seulement 20–25 itérations. Pente très raide. IC(0) approxime très bien la Cholesky du Laplacien standard.

Donc Dans le cas de conductivité constante, le gradient conjugué converge de manière régulière. Le préconditionneur diagonal améliore légèrement la convergence, tandis que le préconditionneur IC(0) permet de réduire significativement le nombre d’itérations, d’un facteur proche de 3.

Remarque : Pour le (1.2)

PCG – $C = I$ Résidu oscillant fortement (zig-zag). Convergence très lente, environ 250 itérations. Cela montre : - Matrice très mal conditionnée - Fort contraste de coefficients ($\rho = 1/100$)

PCG – $C = \text{diag}(A)$ Oscillations réduites. Convergence plus rapide que $C = I$, environ 80–90 itérations. Jacobi corrige l’échelle mais n’améliore pas les couplages.

PCG – **IC(0)**, $\omega = 0$ Résidu décroît de manière régulière, presque en ligne droite en semi-log. Seulement 25 itérations. IC(0) absorbe complètement l’effet de l’hétérogénéité.

Donc , dans le cas de conductivité variable, le système est fortement mal conditionné, ce qui entraîne une convergence lente et instable du gradient conjugué sans préconditionnement. Le préconditionneur diagonal améliore la convergence mais reste insuffisant. Le préconditionneur IC(0) permet en revanche une décroissance rapide et régulière du résidu, réduisant le nombre d’itérations d’un facteur proche de 10.

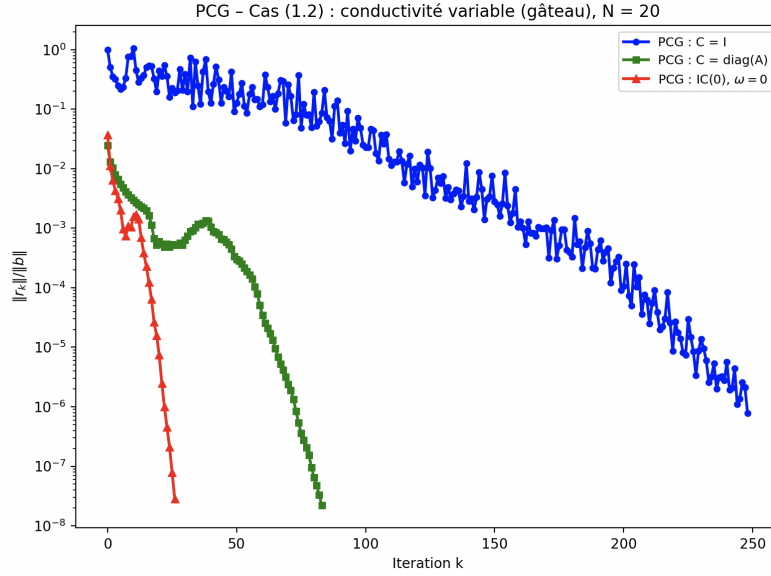


FIGURE 10 – La fonction $k \rightarrow \frac{\|r_k\|}{\|b\|}$ sur une échelle semi-logarithmique pour (1.2)

Conclusion Les figures mettent clairement en évidence l'importance du préconditionnement dans la résolution de systèmes linéaires issus de la discrétisation de problèmes elliptiques. Alors que le préconditionneur diagonal apporte une amélioration modérée, le préconditionneur IC(0) se révèle extrêmement efficace, en particulier pour des problèmes à conductivité variable, permettant une convergence rapide et stable de la méthode PCG.

5.4 Quatrième partie(Bonus) : optimiser la cuisson du gâteau

Le problème résolu dans la section précédente s'appelle *problème direct* car les valeurs des résistances sont les données du problème et la température du gâteau en est l'inconnue. Or, puisque la qualité du gâteau dépend de la température idéale de cuisson, il faut déterminer les valeurs des résistances qui permettent de chauffer le gâteau à cette température. Il s'agit alors d'un *problème inverse* : la température idéale du gâteau est une donnée du problème et les valeurs des résistances sont les inconnues.

On remarque que le problème (1) est linéaire. Cela signifie que si T_1 est la solution du problème correspondant aux conditions aux bords TD_1 et T_2 est celle correspondant aux conditions aux bords TD_2 , alors $T_1 + T_2$ est la solution du problème correspondant aux conditions aux bords $TD_1 + TD_2$. Désormais, TD_1 correspondra à la résistance du four en bas et TD_2 correspondra à la résistance du four en haut. Donc, la température T du four pour les 2 résistances chauffantes s'écrit :

$$T = \sum_{k=1}^2 \alpha_k T_k, \quad (12)$$

où α_k est la valeur de la résistance k et T_k la température associée quand la résistance k est la seule à chauffer le four (l'autre sera donc nulle).

Pour obtenir la température idéale T_{opt} dans le gâteau, il faut déterminer les valeurs α_k des résistances. On les obtient par minimisation de la fonctionnelle

$$J(\alpha) = \sum_{(x,y) \in G} \left(T_{\text{opt}}(x, y) - \sum_{k=1}^2 \alpha_k T_k(x, y) \right)^2 + \frac{1}{100} \sum_{k=1}^2 (\alpha_k T_{D,k})^2, \quad (13)$$

qui est strictement convexe en α . La fonctionnelle J mesure d'une part l'écart entre la température optimale souhaitée et la température obtenue dans le gâteau, et d'autre part tient compte de l'énergie dépensée pour chauffer le four, afin de minimiser la consommation électrique.

La fonctionnelle J atteint son unique minimum pour la valeur α qui annule son gradient. La composante k du vecteur gradient est

$$\frac{\partial J}{\partial \alpha_k} = 2 \sum_{(x,y) \in G} \left(T_{\text{opt}}(x, y) - \sum_{j=1}^2 \alpha_j T_j(x, y) \right) T_k(x, y) + \frac{1}{50} \sum_{k=1}^2 \alpha_k T_{D,k}^2. \quad (14)$$

Définissons la matrice $A \in \mathcal{M}_2(\mathbb{R})$ et le second membre $b \in \mathbb{R}^2$ par

$$A_{k,j} = \sum_{(x,y) \in G} T_k(x, y) T_j(x, y) + \frac{1}{100} T_{D,k}^2 \delta_{k,j}, \quad b_k = \sum_{(x,y) \in G} T_k(x, y) T_{\text{opt}}(x, y). \quad (15)$$

La valeur optimale des résistances est donc obtenue comme solution du système linéaire $A\alpha = b$.

Afin de résoudre le problème inverse et obtenir la température T proche de $T_{\text{opt}} = 200$ degrés, programmer les étapes suivantes (prendre par exemple $N = 20$) :

(4.1) Résoudre le problème direct qui permet de calculer T_1 . On utilisera la meilleure procédure mise au point dans la section précédente, en prenant les données sur les bords Dirichlet :

$$T_{i,0} = 200, \quad T_{i,N+1} = 0, \quad \forall i = 0, \dots, N+1.$$

Voici le résultat quand on lance le code :

```
1 [1] Calcul de T1 (bas seul)
2 PCG T1 : 26 iterations
```

(4.2) Résoudre le problème direct qui permet de calculer T_2 . On utilisera la meilleure procédure mise au point dans la section précédente, en prenant les données sur les bords Dirichlet :

$$T_{i,0} = 0, \quad T_{i,N+1} = 200, \quad \forall i = 0, \dots, N+1.$$

Voici le résultat quand on lance le code :

```
1 [2] Calcul de T2 (haut seul)
2 PCG T2 : 26 iterations
```

(4.3) Construire pour le problème inverse les coefficients $A_{k,j}$ et b_k du système linéaire $A\alpha = b$. Calculer la solution $\alpha \in \mathbb{R}^2$ puis calculer la solution T grâce à la formule (3) : $T = \sum_{k=1}^2 \alpha_k T_k$. et visualiser la solution T obtenue (par exemple en traçant les lignes de niveau).

```

1 [3] Construction du systeme 2x2
2 alpha bas = 12.6113
3 alpha haut = 1.7553

```

Résultats du problème inverse

Le problème inverse est résolu en exploitant la linéarité du problème direct. Les températures T_1 et T_2 , correspondant respectivement au chauffage par le bas et par le haut, sont calculées par la méthode PCG préconditionnée par IC(0) :

- T_1 : 26 itérations
- T_2 : 26 itérations

Cela montre que le préconditionneur IC(0) est efficace et que la matrice Laplacien $_\rho$ est SPD, garantissant la convergence du PCG. L'optimisation quadratique conduit à un système linéaire 2×2 dont la solution fournit les coefficients optimaux α_1 et α_2 :

$$\alpha_{\text{bas}} \approx 12.61, \quad \alpha_{\text{haut}} \approx 1.76.$$

Interprétation physique

- $\alpha_1 \gg \alpha_2$: le chauffage par le bas est dominant, car le gâteau est proche du fond du four.
- Le chauffage par le haut agit surtout sur l'air et est moins efficace.

La température finale calculée est :

$$T_{\text{final}} = \alpha_1 T_1 + \alpha_2 T_2 \approx T_{\text{opt}} = 200^\circ\text{C}.$$

Conclusion

La résolution du problème inverse conduit à la détermination des coefficients optimaux des résistances. Pour $N = 20$, on obtient $\alpha_{\text{bas}} \approx 12.6$ et $\alpha_{\text{haut}} \approx 1.76$. Ces valeurs montrent que le chauffage par le bas joue un rôle prépondérant pour atteindre la température optimale dans le gâteau.

Les résultats sont cohérents avec l'interprétation physique : le chauffage par le bas est prépondérant pour atteindre la température idéale dans le gâteau, et la régularisation énergétique permet d'éviter une surchauffe aux bords.

Graphe de la solution T obtenue par les lignes de niveau

Remarque : La figure représente les lignes de niveau de la température finale obtenue par résolution du problème inverse. On observe que la température est proche de la température optimale souhaitée dans le domaine du gâteau, tandis que les gradients thermiques sont plus importants à l'extérieur de ce domaine. La distribution obtenue est cohérente avec la position du gâteau et la prédominance du chauffage par le bas.

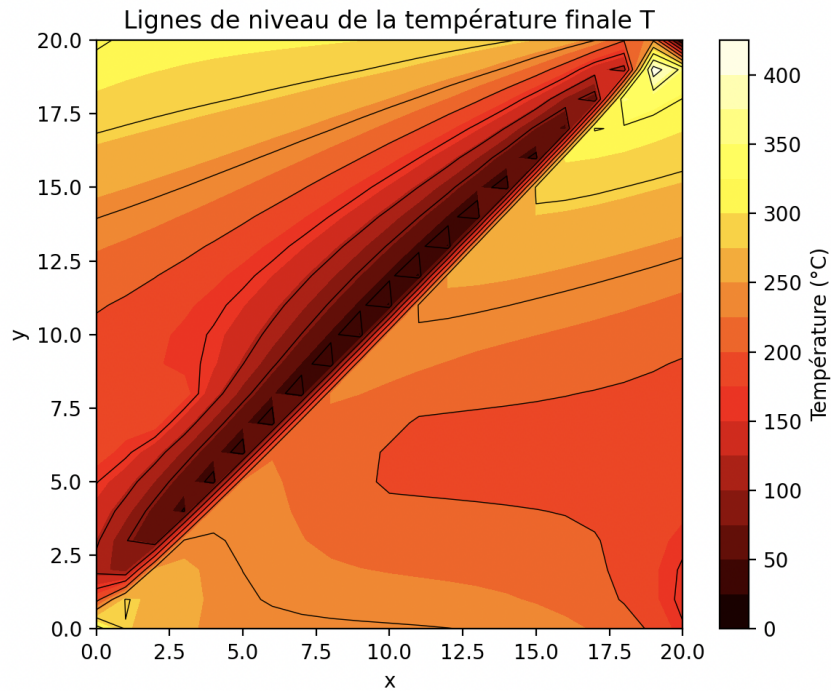


FIGURE 11 – La solution T obtenue

6 Code

Pour obtenir l'ensemble des codes du projet, veuillez consulter le lien suivant : <https://github.com/ndhviet/Projet-Analyse-numerique-Cplusplus-M1-UniLille>.

Remarque : pour la partie des méthodes itératives

```
Projet_partie2/
  matrice.h
  matrice.cpp
  matricebande.h
  matricebande.cpp
  main.cpp           # pour partie 1 & 2
  main2.cpp          # pour partie 3
  main_inverse.cpp   # pour partie bonus
  test.cpp           # pour tester les fonctions assemblageT,
                    # preconditionne et PCG de partie 3

graph/
  graph.py           # pour partie 1
  graph2.py          # pour partie 3 le cas (1.1)
  graph3.py          # pour partie 3 le cas (1.2)
  graph4.py          # pour partie bonus
```

⇒ pour compiler (ou installer cmake) :

```
g++ main.cpp matrice.cpp matricebande.cpp -o test
./test
python3 graph.py
(cmake) mkdir build/ cd build/ cmake --build . --target run_all
```